

FarmersBot

Table of Contents

Table of Contents	1
Introduction (717/1000 words)	4
Motivation	4
Project Aims and Objectives	5
Literature Review (2396/2500 words)	6
Similar work	6
Plantix (Plantix, n.d.)	6
Climate FieldView (Climate.com, 2025)	7
AGRIVI (AGRIVI, 2024)	8
Xarvio Field Manager (Xarvio.com, 2025)	9
Techniques and Methods	11
Crop Growth Simulation	11
Crop Growth Analysis	11
Nutrient Deficiency Identification	11
Crop Disease and Pest Diagnosis Tool	12
Intent Handling	12
Chatbot	13
Design (1982/2000 words)	15
Domain and Users	15
Project Overview	15
Frontend Design	17
Backend Design	18
Contingency Plan	22
Evaluation	22
Implementation (2305/2500 words)	23
Backend Development	23
Model Testing and Selection	23
Crop Growth Simulation	23
Data Collection	23
Soil Data	23
Site Data	23
Crop Data	24
Agromanagement Data	24
Testing	24
Crop Growth Analysis	25
Testing	25
Fine-Tuning	26
Intent Handling	27
Preprocessing	27

Testing	28
FineTuning	29
Chatbot	30
Text Generation	30
Speech Recognition	31
Speech Synthesis	31
Server Coding	32
Frontend Development	33
Evaluation (1829/2500 words)	37
Heuristic Evaluation	37
User Testing - Model Performance	37
Test Plan	38
User recruitment	39
Testing and Data Collection	39
Analysis	40
User Testing - Application	42
First Iteration	42
Testing	42
Second iteration	45
Testing	45
Evaluation Based on Objectives	46
Conclusion (899/1000 words)	47
Reference List	49
Appendices	60
Appendix A	60
Listing A.1	60
Listing A.2	61
Listing A.3	62
Listing A.4	63
Listing A.5	64
Listing A.6	64
Appendix B	67
Listing B.1	67
Appendix C	68
Listing C.1	68
Listing C.2	69
Appendix D	70
Listing D.1	70
Appendix E	72
Listing E.1	72
Appendix F	74
Listing F.1	74
Appendix G	74
Listing G.1	74

Listing G.2	75
Listing G.3	76
Listing G.4	76
Listing G.4: handle_image function	77
Listing G.5	77
Listing G.6	78
Listing G.7	79
Listing G.8	80
Appendix H	81
Listing H.1	81
Listing H.2	85
Listing H.3	85

Introduction (717/1000 words)

FarmersBot builds on the first Artificial Intelligence (AI) template titled “Orchestrating AI models to achieve a goal” found on page 19. The aim of the project is to combine multiple AI models to create a chatbot that assists Kenyan farmers in their day-to-day tasks. The main features include:

- Crop growth simulator - based on the location of the farm, season, soil conditions and the type of crop, the application can simulate an expected crop growth cycle. The simulator allows the farmer to predict potential yield and water usage allowing them to make an informed decision on optimal planting times.
- Crop growth analysis - this system leverages Computer Vision (CV) models to detect and classify nutrient deficiencies, diseases and pests using crop pictures. Based on the detected issues, a prescription is created using a treatments database.
- Audio-based interaction support - to make the application accessible to illiterate farmers who may be unable to understand written text, the output system will include audio conversion models.
- Offline access - most farmers in Kenya live in rural areas where internet connectivity is oftentimes limited. To ensure this user-base is not blocked out from accessing the application, most of the features are usable offline.

Motivation

This project falls under the agricultural domain. Agriculture is arguably the main building block of Kenya's economy (Obudho, 2025). Despite this, there exist limited digital tools accessible to farmers, especially in underdeveloped, remote areas. The main motivation for this project was my mother, who is a farmer in a rural part of Kenya. By observing her day-to-day struggles and looking into academic research, I was able to identify critical areas where farmers would benefit from technological advancements:

- A study found that most farmers incur heavy losses due to sub-optimal crop management strategies (Lone et al., 2021). Sometimes farmers will be unaware of seasonal impacts to crops resulting in poor yields.
 - A solution would involve providing farmers with a tool that can simulate a typical growth cycle based on their location and the type of crop. This would allow farmers to know what the expected yield is and whether planting their intended crop is feasible.
- Factors such as nutrient deficiencies, pests and diseases impact the physiological state of a plant during its development, which in turn influences the yield of a crop at the end of the planting season. Kalaji et al (2014) found that nutritional deficiencies in maize resulted in a 70% negative change in its photosynthetic machinery lowering yield. Research also suggests that early detection and treatment can reverse these detrimental changes (Arnon & Stout, 1939), thus there is a need for tools that can provide this early detection.
 - FarmersBot offers a CV-based analysis tool allowing farmers to identify and treat nutritional deficiencies, pests and diseases in their crops early. This would prevent irreversible damage to crops which could cause severe losses.

- According to Reeves et al (2024), lack of literacy impedes the ability for rural farmers to get educated on modern agricultural practices, forcing them to rely on traditional approaches that often produce low yields.
 - The application operates around a chatbot with speech synthesis and recognition ability. This allows farmers to orally communicate with the system instead of being forced to type in input.
- Verkaart et al. (2018) found that most farmers in Kenya live in rural areas. Most rural parts of the country have limited cellular connection (Ict.go.ke, 2017), effectively blocking a large number of Kenyan farmers from accessing digital tools.
 - A solution would be ensuring all the features offered in the system are fully accessible offline.

Project Aims and Objectives

The main aim of this project is to provide Kenyan farmers with an easy to use chatbot assistant that offers advice on the various tasks that go into farming. The objectives include:

- Design and develop a user-friendly, accessible chatbot interface with support for both audio and text-based interaction and image upload.
- Develop a backend server linked to the frontend with the required logic to process the input received from the frontend, decide which model should receive the input, if the input requires preprocessing and how to send the generated output to the frontend.
- Test and select models that perform the following functions:
 - Crop growth simulation
 - Crop growth analysis tools
 - Speech synthesis.
 - Speech understanding.
- User-centered evaluation of the final product with iterative improvements prior to deployment.

Literature Review (2396/2500 words)

Similar work

An application similar to FarmersBot has not been fully implemented. Nevertheless, some existing applications have implemented certain features. These are discussed in this section.

Plantix (Plantix, n.d.)

Plantix is one of the most popular applications in the field of AI-based crop disease and pest diagnosis. It boasts impressive features including:

- Image-based diagnosis supported by AI models and a large database of crop diseases and pests

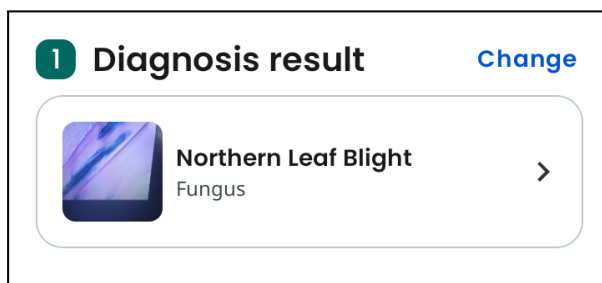


Figure 1: Image-based diagnosis on Plantix

- Treatment recommendations and advice on preventative measures.

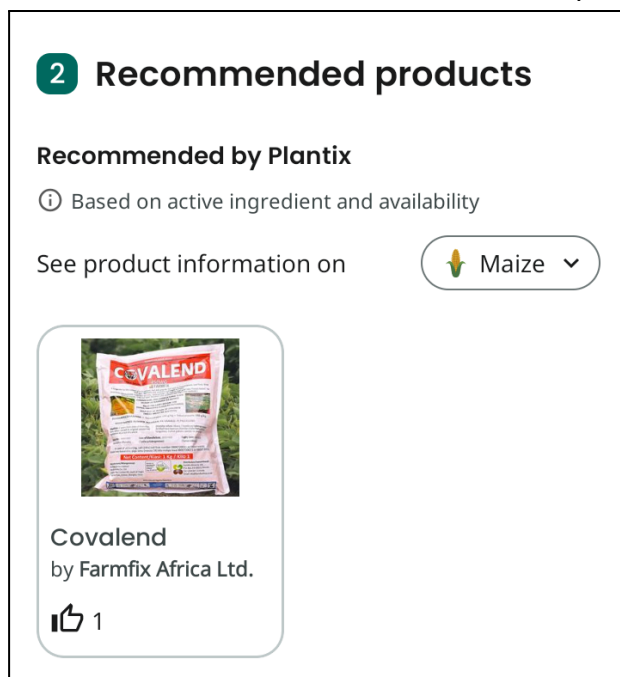


Figure 2: Plantix product recommendations

- Geolocation capabilities allowing for localized diagnosis and alerts to other local farmers on potential outbreaks.

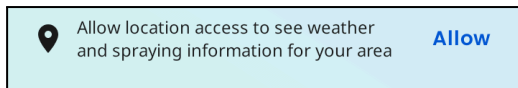


Figure 3: Plantix geolocation capabilities

Plantix still suffers from a few limitations such as:

- A lack of coverage for less common diseases and pests, especially local varieties native to certain parts of the world.
- Dependence on image quality where poor angles, blur or background interference results in an inability for the app to function as expected. This impacts farmers who use cheap smartphones with minimal image capturing capabilities.
- Lack of offline support for most features which impacts farmers living in remote areas with low internet connectivity.

Climate FieldView ([Climate.com](https://climate.com), 2025)

Climate FieldView helps farmers understand how their crops are faring and offers suggestions on what decisions to make during the season. It consists of two mobile applications alongside a website. These are:

- FieldView Drive - this is a small device installed in whatever machinery the farmer is using as they work on the field. It records the various actions performed alongside other information such as Global Positioning System(GPS) location and transmits the data collected to the FieldView app via bluetooth.



Figure 4: FieldView Drive

- FieldView app/website - used for viewing and analysis of data collected in the field. It offers visualizations of the data and uses statistical models to analyze factors such as crop yields. This information can be used to generate prescriptions and predict the response of different zones to the prescriptions.



Figure 5: Analysis on the fieldview app

The application does have a number of limitations which will be addressed by my project including:

- It does not cater to small scale farmers as the FieldView Drive device is designed to be attached to heavy farm equipment which are not used in small scale farming. A simpler smartphone alternative would better serve small-scale farmers.
- The scouting tools rely on previous data collected from the field to create prescriptions for crop, weed and pest treatments. While this is useful for repeat crops, farmers who are constantly switching their produce would better benefit from an image-based crop disease and pest diagnosis.
- Analysis on the FieldView application is dependent on having an internet connection, which is not always available for farmers in developing countries thus creating a barrier to entry.

AGRIVI (AGRIVI, 2024)

This is a digital farm management system that helps track activities in the farm, issues alerts for crop pest and disease risk and uses satellite-based remote sensing to look at vegetation indices that help to keep track of crop and field conditions. A major selling point of the platform is Ed, a Whatsapp-based chatbot that serves as an agricultural advisor. It uses a backlog of agronomic knowledge to respond to questions that farmers may have on their crops.

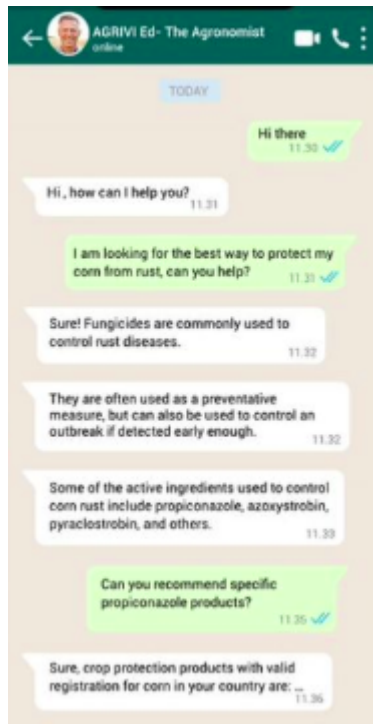


Figure 6: Agrivi's Ed

Some limitations of AGRIVI's Ed system include:

- Ed does not currently support image-based advice and relies on the farmer's description of the challenge they may be facing which may lack key details.
- Ed simply responds to a question and does not use knowledge of the specific process that the farmer has undergone so far to tailor the response. A better system would offer advice based on the farmer's specific journey.
- Since Ed is a whatsapp-based chatbot, it is reliant on an internet connection and is thus inaccessible to remote farmers with unreliable connections.

Xarvio Field Manager ([Xarvio.com](https://xarvio.com), 2025)

Xarvio perform field management functions including:

- Field monitoring - recording and analysis of field data such as biomass and soil maps



Figure 7: Xarvio Field Monitor (Xarvio.com, 2025)

- Seeding - data-based planning of which parts of the field to seed

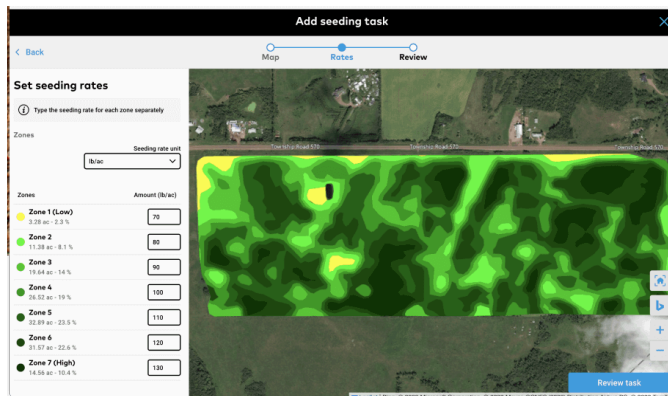


Figure 8: Xarvio Seeding tool (Xarvio.com, 2025)

- Crop nutrition analysis - management of field-wide fertilizer usage based on crop needs at the time

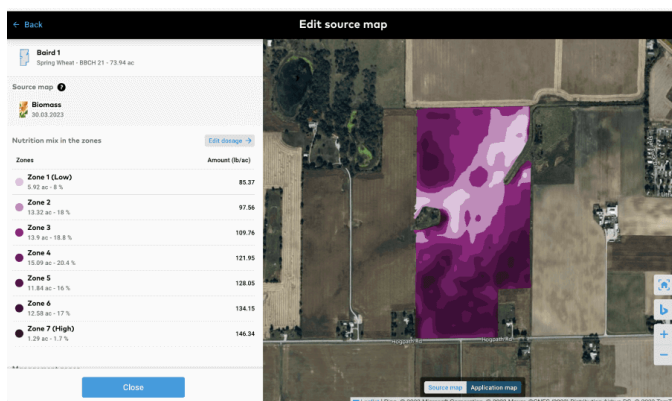


Figure 9: Xarvio Crop nutrition guide tool (Xarvio.com, 2025)

- Crop protection - keeps track of pest and disease risks and offers suggestions on preventative measures.

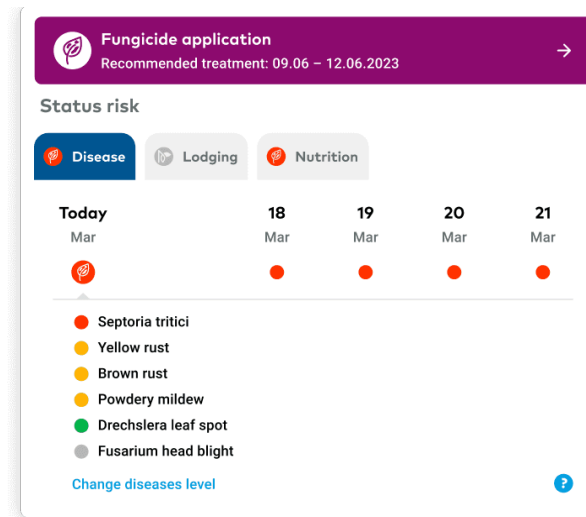


Figure 10: Xarvio Crop Protection tool (Xarvio.com, 2025)

The crop nutrition analysis protection systems are similar to the intended features for FarmersBot. On Xarvio, several limitation with these features were noted:

- Nutrition analysis and crop protection are based on modeling and timing and rely on pre-existing knowledge. This is not always accurate due to changes in climatic

conditions or unexpected outbreaks. A diagnostic tool based on image analysis would better suit such edge cases.

- Xarvio relies on advanced farming equipment such as sensors which may not be available to most small scale farmers in Kenya.

Techniques and Methods

Crop Growth Simulation

One of the key features of my application is a crop growth simulator. The creation of a crop growth simulation model has been a major field of research since the 1980s (Bhatia, 2014). A number of regression models were developed during this time to make predictions on potential agricultural yield and the factors going into the process of crop growth (Singh, 1994). Regardless, these models mostly focused on technological trends and excluded other variables such as climatic, edaphic and socioeconomic factors (Singh, 1994). A major issue with such models is the reliance on regression models which require the relationship between the dependent and independent variables to be constant. This is not true for most factors affecting agroecosystems such as climatic conditions (Ingram and McCloud, 1984). On top of that, as I see it, technological trends are oftentimes insignificant factors influencing agricultural productivity.

Different types of modelling strategies are required for different types of problems. In the case of crop growth simulators, most modern simulators are knowledge-based (Jame and Cutforth, 1996) and integrate “crop physiology, weather parameters, soil parameters and management practices to simulate growth and yield of crops” (Bhatia, 2014, p. 1). Dynamic modeling, involving utilizing mathematical representations to characterize agroecosystems, has proven to be an effective tool in simulation of crop growth cycles as it allows for the integration of multiple processes (Zhang et al., 2002). Since agroecosystems are heavily dependent on multiple factors, most of which are constantly changing (Ingram and McCloud, 1984), I think a dynamic model with the ability to accommodate all these features into its predictions would better suit the purposes of the crop growth simulation system.

Crop Growth Analysis

Precision Agriculture (PA) has been widely adopted in most agroecosystems with one of its major factors being crop growth analysis. In FarmersBot, it involves these features:

- Nutrient deficiency detector
- Crop disease and pest diagnosis tool

The combination of these two features into one tool is novel to FarmersBot. However, research into the individual components has been done previously.

Nutrient Deficiency Identification

Knowing when crops lack certain nutrients is crucial for farmers. Most use guesswork or estimates to identify deficiencies (Saha et al., 2021). This method is prone to error due to the farmer’s limited knowledge and bias (Cao et al., 2022). Most farmers were forced to rely on

it, though, since early tools relied on soil testing (Dimkpa et al., 2017) or leaf tissue analysis (Bouma, 1983) in the lab, which was too expensive and time consuming (Nikitha, Prabhanjan & Sathyanarayan, 2025).

Since these early tools were unavailable for ordinary farmers, a gap for cheap and time efficient tools was created. Lots of research has gone into filling this gap. Some techniques include leaf analysis combined with complex interpretation techniques such as Critical Level (CL) (Luciel et al., 2016), using chlorophyll meters such as Dualex to determine chlorophyll content (Muñoz-Huerta et al., 2013) or relying on remote sensing methods to estimate certain morphological plant properties and determine nutrient content (Stroppiana et al., 2009). Nonetheless, each of these proposed solutions has a number of limitations. Leaf analysis is extremely time consuming and requires numerous additional interpretation methods (Cunha et al., 2016). Chlorophyll meters are not sufficiently sensitive to a number of different treatments (Perry and Davenport, 2007) and remote sensing techniques require sophisticated equipment and are only efficient on a large scale (Muñoz-Huerta et al., 2013).

A solution to all these limitations has been combining ML techniques with Computer Vision (CV) (Barbedo, 2019). The use of digital images to capture various symptoms (Chen et al., 2014) combined with regression models, often linear (Barbedo, 2019), has proven to be the most accurate strategy to perform nutrient deficiency identification and classification based on proximal images. Modern advances in smartphone photography capabilities (Lingyan et al., 2018) make this the most effective strategy for FarmersBot.

Crop Disease and Pest Diagnosis Tool

Crop pests and diseases prove to be one of the largest contributors to poor yield in agroecology. When it comes to automated crop pest and disease identification based on images, there have been significant advancements in research since 2024 (Lu, Tan and Jiang, 2021). Most traditional methods relied on simple ML models, specifically supervised models that require training and testing data (Wang et al., 2025). Several steps would be involved in working with these models culminating in feeding data to simple ML algorithms such as SVM (Zhang et al., 2014) and Principal Component Analysis (PCA) (Bo et al., 2009). While these models have proven to be rather accurate (Bo et al., 2009), it has been found that these models are unable to be generalized and thus cannot be applied to larger contexts (Wang et al., 2025).

Most modern models rely on Deep Neural Networks (DNN). Certain models, such as YOLO-GBS, incorporate an attention head and mask for higher accuracy in detection and classification (Hu et al., 2023a). Convolutional Neural Networks (CNN) have also shown great promise in this task (Mohanty, Hues and Marcel, 2016). In addition to the high accuracy, CNNs are also largely generalizable in comparison to ML algorithms (Lu, Tan and Jiang, 2021). For this reason, a CNN-based model would be best suited for the crop pest and disease recognition system.

Intent Handling

Intent handling is the process by which conversational agents map user prompts to specific actions that generate a response that is relevant to the original prompt. It is a subset of text

classification that serves as a critical first-level task in chatbots, especially ones having multiple models working together, as it ensures that the correct model is selected for a given task (Pawlik et al., 2022). Plenty of research has gone into creating optimal intent handling algorithms. Early techniques largely involved rule-based algorithms (Dowding et al., 1993) which later were phased out with more modern models using statistical or deep learning algorithms (Liu et al., 2019).

Traditional rule-based techniques involved specifying a set of patterns and mapping different types of user input to them (Dowding et al., 1993). A major limitation arose due to their reliance on grammar rules which made them rigid and impossible to scale over multiple domains (Sim et al., 2025). This caused them to be replaced by statistical algorithms which use Machine Learning (ML) algorithms on manually annotated prompts (Hefny et al., 2020). According to Hamroun and Gouider, (2020), ML techniques have become the industry standard for most Natural Language Understanding (NLU) tasks including intent handling. Recently, deep learning techniques, mostly using Convolutional Neural Networks (CNN), have emerged. These models are better able to capture semantic and syntactical nuances compared to the other techniques (Kanani & Sanghani., 2025). However, they often underperform when compared to statistical algorithms (O Sullivan., 2018, Kanani & Sanghani., 2025, Schuurmans & Frasincar., 2019). Therefore, I intend to use statistical algorithms for my project.

Hassan et al (2022) performed a comparative analysis of some of the popular statistical algorithms in text classification. The study used two datasets; one from the Internet Movie Database (IMDb) which consists of data on various movies and celebrities and SPAM dataset containing emails labeled as either ham or spam. The algorithms tested were Support Vector Machine (SVM), k-Nearest Neighbor (K-NN), Logistic Regression (LR), Multinomial Naïve Bayes (MNB), and Random Forest (RF). Various preprocessing steps were performed on the data including converting the documents to plain text and lowercase, removal of stop words and filler words and lemmatization after which the text was vectorized using the Bag-of-words (BOW) and Term Frequency Inverse Document Frequency (TF-IDF) models. Due to the skewed nature of the SPAM dataset, the best evaluation metrics were found to be precision, recall and F1-scores. The researchers found that SVM and LR were the best performers with accuracies of 85.5% and 85.9% respectively. I plan to repeat this study with fine-tuned hyperparameters, which was a limitation noted by the researchers. The best performing model can then be fine-tuned to be specific to intent handling for FarmersBot.

Chatbot

Major leaps in Natural Language Understanding (NLU) capabilities have caused a rapid increase in the popularity of chatbots (Lin, Huang and Yang, 2023). Despite this, research into chatbot creation goes back to the 1950s with Alan Turing's proposal of the Imitation Game whose goal was to figure out if machines can think (Buhari et al., 2023). The aim was to see how well an artificial chatbot could fool a human into believing they were interacting with another human (Buhari et al., 2023).

Significant advancements in the development of such intelligent machines have been made since those early days. Various techniques have been proposed such as using simple

pattern matching. A famous example of a system that used this strategy was ELIZA, a basic chatbot that used regular expressions combined with a few decomposition rules to mimic the style of a Rogerian psychotherapist (Weizenbaum, 1976). More modern systems, in comparison to ELIZA, strive to emulate reasoning. This was commonly done using NLU and ML techniques such as in Chandel's (2025) healthcare chatbot which was based on SVM and Decision tree models. While these models could achieve rather high accuracy, they suffered from bias obtained from the training data (Chandel, 2025).

To address the shortcomings of ML-based models, modern day transformer models use attention masking (Vaswani, 2017). Such models consist of an encoder, which deciphers input, and a decoder, which generates the output (Yigit and Bayraktar, 2025). When used in Natural Language Processing (NLP) tasks, they have been shown to have accuracies as high as 83.28%, significantly outperforming other models in the same tasks (Rafel et al., 2020). Based on the literature, using a transformer model for the NLU tasks in this project would be the best option.

Design (1982/2000 words)

Domain and Users

FarmersBot's selected domain is Kenyan agriculture which is the leading contributor to Kenya's GDP (Obudho, 2025). Local agriculture also contributes to 92% of the food consumed (Statista, 2023). Since most agricultural activity is small-scale (Obudho, 2025), yield improvements for small-scale farmers not only contribute to the national economy, but also improve the country's food security. This creates a need for technological tools that help increase yield for such farmers.

The use of digital tools among small-scale Kenyan farmers, which would help increase yield (Coggins et al., 2022), is still limited (Juma, Nyangena & Yesuf, 2022). According to Ambali, Areal and Georgantzis (2021), this is due to poverty in areas where most agricultural activities are undertaken. This highlights the need for low-cost solutions for rural farmers. Besides poverty, Michieka (2011) found that lack of literacy also hinders their ability to use automated tools, creating a need for an audio-based chatbot. Another consideration is the need for internet connection to use most digital tools. Since most of rural Kenya has low internet access (lct.go.ke, 2017), rural farmers cannot use these tools, hence why an offline solution is necessary.

There is clearly a need for a free, offline, agricultural tool for Kenyan farmers. Since 40% of Kenyans work in rural, small-scale agriculture (Centralbank.go.ke, 2024), this demographic is suitable for the project's user-base. This demographic can be categorized as:

- Illiterate farmers - supported through audio-based interaction
- Literate farmers - supported through text-based interaction

The different interaction options also create accessibility for individuals living with disabilities.

Project Overview

FarmersBot is an AI-powered digital assistant for small-scale, rural, Kenyan farmers. The system architecture includes:

- Frontend - receives user input and output in audio and text format
- Backend - handles the application logic
- Database - stores user details.

As shown in the system architecture in Figure 11, the user's input is first converted to text if necessary. It is then passed to the intent handling model which determines the user's aim and passes the input to the relevant function. Results are converted to audio, if required, and returned to the frontend.

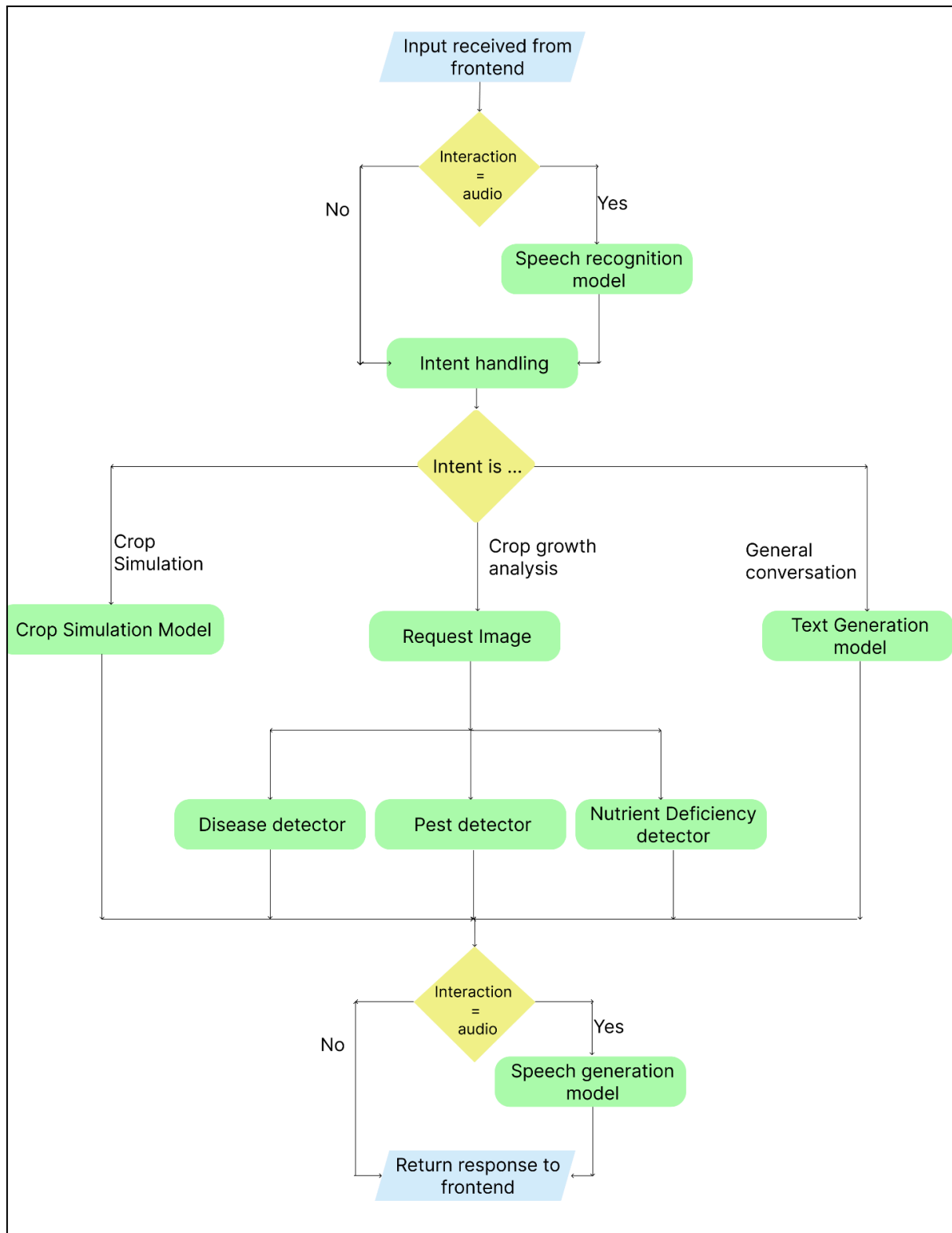


Figure 11: System architecture

The frontend uses Python's Kivy library, and the backend, Python with FastAPI for frontend integration. Additional libraries include:

- Pandas
- SQLAlchemy
- Numpy

- PyTorch
- Tensorflow
- Scikit-learn
- Huggingface_hub

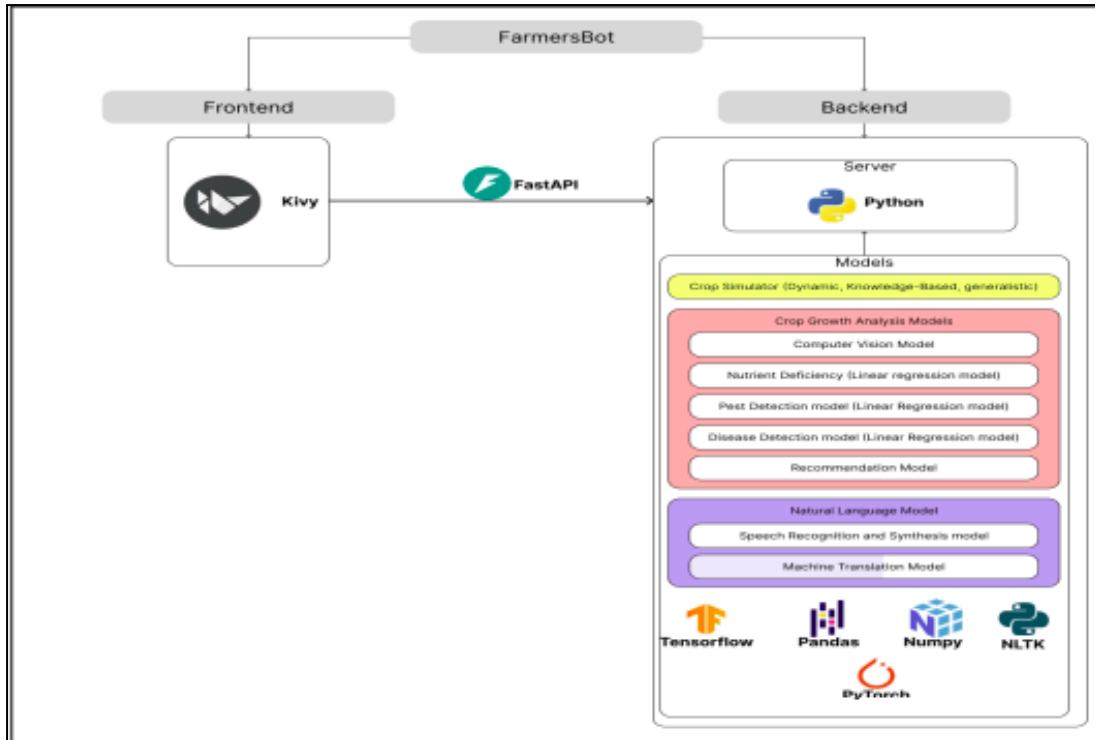


Figure 12: Application techstack

Frontend Design

A Born-Accessible development framework is used for the frontend since it is both cost and time effective (Lazar, 2023). The interface includes:

1. Text input keypad.
2. Microphone for audio-based input creating accessibility for visually-impaired and illiterate users.
3. Camera for the image-based crop growth analysis models.
4. Text-based output panels with a font size of 14 points as per WCAG recommendations (W3C, 2024) and Helvetica font style which is most conducive for a diverse user-base (Wallace et al., 2022)
5. Audio-based output panel for users with visual impairments and illiteracy.

The color scheme uses the 4:5:1 contrast ratio recommended by WCAG (W3C, 2024) and the primary color is green to match the application's agricultural theme.

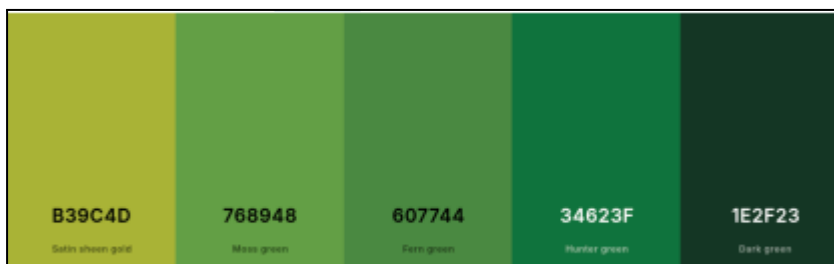


Figure 13: Color scheme (Bianchi, 2025)

An iterative approach was taken to prototyping, using real users with both qualitative and quantitative questions. Participants were gathered through opportunistic sampling from farmers in Kenya's Kajiado county. Considerations were made to ensure that both literate and illiterate users alongside users with disabilities were included. The data collection methods were:

- a. Questionnaires for literate users
- b. Interviews for illiterate users

Based on the survey's results, the wireframe in Figure 14 was developed, consisting of audio and text-based interaction pages, a signup/login page and a settings page.

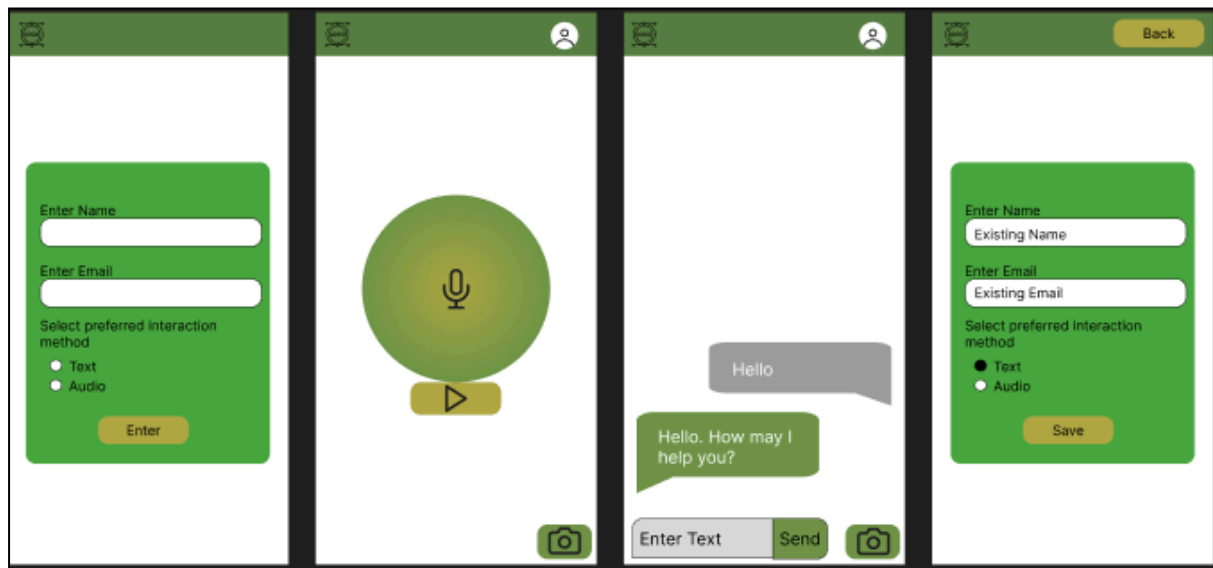


Figure 14: Final hi-fi prototype

Backend Design

Most of the backend development process involves testing and selecting models for the following functions:

- **Intent handling** - this selection replicates Hassan et al's (2022) paper as per the literature review.
 - **Models:** The statistical algorithms, Support Vector Machine (SVM), k-Nearest Neighbor (K-NN), Logistic Regression (LR), Multinomial Naïve Bayes (MNB), and Random Forest (RF) will be tested using their Scikit-learn implementations.
 - **Datasets:** unlike the original paper, the models will only be tested on the spam dataset since the IMDb dataset tests for sentiment classification, which is irrelevant to intent handling.
 - **Spam dataset** sourced from University of California, Irvine's (UCI) ML repository (Hopkins et al., 1999), it contains 5,574 messages with binary (1=spam, 0=ham) labels. The messages have been vectorized through the Bag-of-Words (BOW) and Term-Frequency, Inverse-Document-Frequency (TF-IDF) algorithms.
 - **Testing:** As recommended in the paper, tuning hyperparameters will be used through grid search with cross-validation. The test metrics are:

- Accuracy - percentage of correctly predicted labels.

$$Accuracy = \frac{\# \text{ of correctly predicted labels}}{\# \text{ of total predictions}}$$

- Precision - proportion of predicted spam labels that were actually spam to minimize false positives.

$$Precision = \frac{true \text{ positives}}{true \text{ positives} + false \text{ positives}}$$

- Recall - proportion of actual spam labels correctly predicted as such ensuring completeness of results.

$$Recall = \frac{true \text{ positives}}{true \text{ positives} + false \text{ negatives}}$$

- F1-score - provides a harmonic mean of precision and recall. This will be prioritized, to ensure the models are not simply predicting spam by deleting ham emails.

$$F1score = 2 \times \frac{precision \times recall}{precision + recall}$$

- **Fine-tuning:** the selected model needs fine-tuning to support the intent handling labels intended for this project, those being; crop simulation, crop growth analysis and general conversations. This data will be sourced through an exploratory survey amongst rural farmers using interviews to determine how participants phrase prompts for the different tasks.

- **Crop Growth Simulation**

- **Models:** as determined in the literature review, mechanistic models are best suited for this task. Therefore, the models tested are mechanistic including:
 - WOFOST (WORLD FOOD STUDIES) (Wageningen University & Research, 2025).
 - LINTUL (Light INTERception and Utilisation) (Pps.wur.nl, 1989).
 - DSSAT (Decision Support System for Agrotechnology Transfer) (DSSAT.net, n.d.).
 - APSim (Agricultural Production Systems sIMulator) (APSIM, n.d.).
- **Datasets:** the models will be tested on data from Butere, Kakamega Central, Kakamega East, Gilgil, Kuresoi North, Kuresoi South, Bumula, Bungoma Central and Bungoma East subcounties instead of the entire country to save on time.
 - Soil data - sourced from SoilGrids API (Isric.org, 2025). For each subcounty, the bulk density (cg/cm³), clay, sand and silt content(g/kg), and volumetric water content at 10kPa, 1500kPa and 33kPa pressure levels (10⁻³cm³cm⁻³), will be obtained for the top layer (0-5 cm).
 - Crop data - sourced from the Python Crop Simulation Environment (PCSE) (Readthedocs.io, 2018) for maize, sunflower, cowpeas and millet only as these are the most commonly grown crops in the country (Kilimo.go.ke, 2019) with available data in the database. Each crop's data file contains its dry matter distributions (kg), respiration and transpiration rates (CH₂O kg⁻¹ d⁻¹), leaf area indices, carbon dioxide and nitrogen absorption rates (kg CH₂O ha⁻¹ h⁻¹) and effects of temperature and humidity changes on dry weight.
 - Site and agromanagement data - dummy data is used since no field experiments have been conducted. Site data consists of default soil parameters (similar to soil data) and agromanagement includes

planting date, crop name and variety and maximum duration of planting campaign.

- Weather data - sourced from NASA's power point API (Stackhouse, 2024) in a time series format, consisting of weather measurements from 2024 for each subcounty. The main parameters are precipitation levels (mm), humidity (%), temperature (°C) and solar radiation (watts).
- Actual yields (kg/ha) - sourced from the The Kenya National Bureau of Statistics report on agricultural production in 2024 (KNBS, 2024). For each crop mentioned in crop data, the quantity produced in every subcounty will be manually copied from the report and saved in a dataframe.

○ **Testing:**

- Root Mean Square Error (RMSE) - used to quantify the difference between simulated and actual yields. For each subcounty and crop combination, the model will use the soil, crop, site, agromanagement and weather data described above to simulate a typical crop cycle. RMSE will then be calculated as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_{sim,i} - Y_{obs,i})^2}$$

Where $Y_{sim,i}$ = simulated yields, $Y_{obs,i}$ = actual yields and n = number of observations

- **Crop Growth Analysis** - in the literature review, it was planned that separate heads would be tested for the nutrient deficiency, disease and pest detectors then combined with a pre-trained CV-base. However, complete, general purpose object identifiers with CNN-architectures will be tested instead for time-saving and consistency purposes.

○ **Models:**

- Real-Time DEtection TRansformer (RT-DETR) (Zhao, et al., 2024).
- D-FINE small (Peng et al., 2024)
- You Only Look at One Sequence (YOLOS) (Fang, et al., 2021).
- Faster RCNN (Lin, et al., 2017).
- Mask RCNN (He, et al., 2017).

○ **Datasets:**

- The Common Objects in Context (COCO) dataset (Lin et al., 2014), a general object detection dataset obtained through the COCO API and consisting of over 200,000 annotated images with over 1.5 million object instances. Each annotation contains the detected objects' bounding boxes (uses the COCO format [x-top-left, y-top-left, width, height]), area and its name.

○ **Testing:**

- F1-score - calculated using the precision and recall scores from COCOeval's evaluation functions. These scores will be calculated for the different IoU (Intersection over Union) thresholds which is the specific cutoff typically used to calculate a model's correctness. The model with the highest score will be selected.

$$F1score = 2 \times \frac{precision \times recall}{precision + recall}$$

- **Fine-tuning:** the selected model needs to be fine-tuned for the separate tasks. The data used for this includes:
 - Disease and nutrient deficiency detection - sourced from the PlantDoc dataset which consists of 2598 data points (Singh et al., 2019).
 - Pest detection - data sourced from the IP102 database which contains over 75,000 images covering around 102 species of plant pests (Wu et al., 2019).

Both datasets contain images and their annotations including bounding boxes (Pascal VOC [x-min, y-min, x-max, y-max] format) and class of the detected issue.

- **Chatbot:**

- **Models:**
 - Text generation - Qwen2.5 (Yang et al., 2025), GPT-2 (Radford et al., 2019), GLM-4.5 (Zeng et al., 2025), MiniMax (Huggingface.co, 2026, b) and ReaderLM-v2 (Jina AI, 2025).
 - Speech synthesis - Kokoro (Huggingface.co, 2024), Kyutai TTS (Zeghidour et al., 2026), Variational Inference with adversarial learning for end-to-end Text-to-Speech (VITS) (Pratap et al., 2023) and Bark (Huggingface.co, 2023).
 - Speech recognition - OpenAI Whisper (OpenAI, 2022), AventIQ AI (Huggingface.co, 2026, a), Vosk (small) (VOSK Offline Speech Recognition API, n.d.) and Wav2Vec2 (ai.meta.com, n.d.).
- **Datasets:**
 - Text generation - sourced from the Chatbot Arena (Lmarena.ai, 2024) which contains over 33,000 prompts-response pairs between various Large Language Models (LLMs) and users with user ratings on the preferred model between two options.
 - Speech synthesis - user testing will be performed for each model with real users.
 - Speech recognition - the accentsDB dataset sourced from Huggingface (Canuma, 2026) consisting of 12,600 audio clips and their transcriptions.
- **Testing:**
 - Text Generation:
 - BERTScore - will measure the semantic similarity between the model's responses and the dataset's reference for each prompt in the Chatbot Arena
 - Word Error Rate (WER) - calculate the percentage of incorrect words in the response as compared to the reference, specifically in relation to insertions and deletions.
 - Toxicity - measure offensiveness in text.
 - Speech Synthesis:
 - The highest ranked model from user testing will be selected.
 - Speech understanding:

- Character Error Rate (CER) - similar to WER but calculations are made on a character rather than word level.
- WER
- Toxicity

Contingency Plan

If the classical models for crop growth analysis fail to achieve an acceptable performance i.e. an F1-score greater than 0.75, a lightweight transformer model like DistilBERT will be used. In case a model's size or latency is found to exceed the device constraints of typical Android and Apple smartphones, it will either be replaced by a smaller version with fewer checkpoints. If none exists, a simpler classifier model alternative will be used in its place. For intent handling tasks, in case the data gathered for fine-tuning is insufficient, a traditional rule-based technique will be used as a fallback to retain basic system functionality.

Evaluation

The entire project will be evaluated using three strategies:

- **Heuristic evaluation:** this will allow for design problems to be identified (Molich & Nielsen, 1990). The identified problems will shape the aims of the rest of the evaluation process.
- **User-testing** - this process will involve studies conducted with real users to test out:
 - **Models** - outside of the automatic evaluation criteria outlined in the model selection process, the models practical usability will be tested through user-testing. The metrics to be evaluated will be determined during the heuristic evaluation.
 - **Application** - the final product's usability will be evaluated using real users based on the results of the heuristic evaluation.

Finally, the overall success of the project will be evaluated against the project's aims outlined in the Introduction section.

Implementation (2305/2500 words)

The development process was split into two stages:

- Backend Development
- Frontend Development

Backend Development

Model Testing and Selection

Crop Growth Simulation

This involves predicting crop development using mathematical representations of plant physiology. Four models were specified for this task in the Design. However, only WOFOST and LINTUL were eventually tested because;

- DSSAT only has FORTRAN implementations making it difficult to integrate into FarmersBot’s Python ecosystem.
- APSim is designed to run with a full .NET runtime and not as part of a separate application.

Data Collection

Soil Data

The original plan was to source this from the SoilGrids API using a `fetch_soil_parameters` function (Listing A.1, Appendix A). However, I was only able to obtain data for three subcounties before the API was taken down. The next method attempted was downloading data from the Kenyan Soil and Terrain Database (KENSOTER) (Wageningen University & Research, 2026) but the data contained too many null values.

	ISOC	SUID	NEWSUID	SoilMapUnit	SOIL1	PROP1	PRID1	SOIL2	PROP2	PRID2	...	PRID7	SOIL8	PROP8	PRID8	SOIL9	PROP9
0	KE	1	KE1	CMu1 [1]	CMu	100	KESYN1	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN
1	KE	2	KE2	ACh1 [2]	ACh	100	KE115/1-12	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN
2	KE	3	KE3	LVv3CMv3FLe5 [3]	LVv	45	KE115/4-47	CMv	40	KE115/4-45	...	NaN	NaN	NaN	NaN	NaN	NaN
3	KE	4	KE4	CMo4ACh4NTr4ACp4 [4]	CMo	30	KE115/2-26	ACh	30	KE115/2-35	...	NaN	NaN	NaN	NaN	NaN	NaN
4	KE	5	KE5	VRe1FLm5 [5]	VRe	85	KE115/4-48	FLm	15	KE115/2-9	...	NaN	NaN	NaN	NaN	NaN	NaN

Figure 15: Dataframe showing most of the values in the dataset were NaN

Finally, I resorted to using a SoilGrid’s Python package (gantian127, 2025) using the `soilgrids_data` function (Listing A.2, Appendix A). Some units were computed using Saxton and Rawls’ (2006) equations.

Site Data

PCSE’s implementation of WOFOST provides a `WOFOST72SiteDataProvider` function that creates dummy site data (Readthedocs.io, 2024). LINTUL data was obtained from Soilgrids using the `soilgrids_site` function (Listing A.3, Appendix A).

Crop Data

WOFOST crop data was directly obtained from PCSE's database through the `YAMLCropDataProvider` model (Readthedocs.io, 2022). As for LINTUL, some of the data was manually obtained from the same database and other parameters were set to defaults.

Agromanagement Data

This was created manually based on templates obtained from PCSE (Readthedocs.io, 2017).

Testing

`test_wofst` (Listing A.4, Appendix A) and `test_lintul` (Listing A.5, Appendix A) functions were implemented that iterate through every crop and location then return simulation results. The RMSE score between the simulated total weight of storage organs (TWSO), and actual yield values, were computed using Scikit-learn's implementation of the RMSE function. The LINTUL model performed significantly worse than WOFOST as seen in Figure 16 and 17.

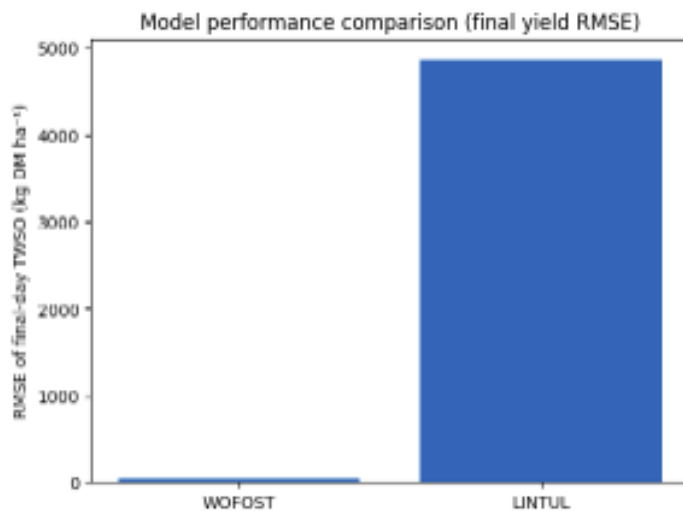


Figure 16: Graph showing the difference between WOFOST and LINTUL

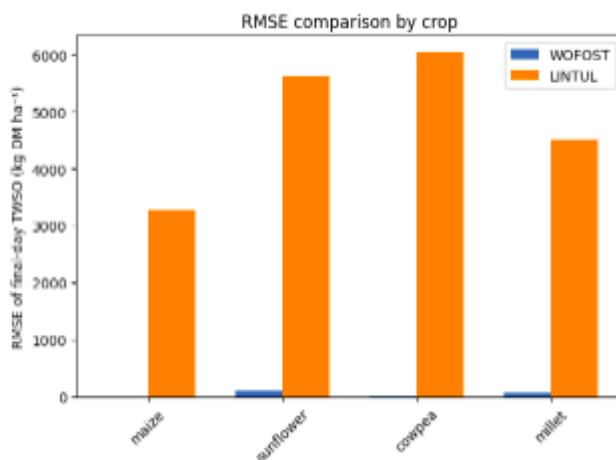


Figure 17: Crop-by-crop comparison of the models

This was likely due to data quality since most of the data used in WOFOST was directly obtained from PCSE while LINTUL data was manually created, with many default values.

Based on these results, WOFOST was selected for use in FarmersBot through a `run_simulation` function (Listing A.6, Appendix A). This function fetches soil data from a local database created from the SoilGrids Python Package, crop data from PCSE's `YAMLCropDataProvider`, and weather data through NASA's power point API. Only crops in the `YAMLCropDataProvider` database are supported and future work should look into obtaining more crop data from other sources.

Crop Growth Analysis

This feature consists of three models:

- Nutrient deficiency identifier
- Crop pest identifier
- Crop disease identifier

The COCO data was downloaded from github instead of using the COCO API for time-saving purposes. The testing process involved running each image from the dataset through the models specified in the Design section and storing predicted annotations in json format (storage was done because the testing process took multiple days due to the large size of the dataset). An example of this process for the Peking model is shown in Listing B.1, Appendix B.

Testing

For each model, the precision and recall scores across the different confidence thresholds at each IoU threshold were obtained using the `coco_eval.eval` function. These scores were used to calculate F1-scores where the maximum score across each IoU threshold's confidence thresholds were selected for comparison. The results are shown in Figure 18.



Figure 18: F1-scores visualization

The results showed that the Peking model outperformed all other models at each IoU threshold and it was thus selected for use in FarmersBot.

Fine-Tuning

Due to limited fine-tuning data, the application will only support plants listed below:

- Disease and nutrient deficiency detection:
 - Tomato - Early blight, Septoria leaf spot, Bacterial spot, Late blight, Mosaic virus, Yellow virus, Mold, Two-spotted spider mites
 - Apple - Apple scab, Apple rust
 - Corn - Common rust, Gray leaf spot, Northern leaf blight

- Potato - Early blight, Late blight
- Grape - Black rot
- Bell pepper - bacterial spot
- Squash - powder mildew
- Pest detection:
 - Rice Pests: Asiatic rice borer, brown planthopper, paddy stem maggot, rice gall midge, rice leaf roller, rice leafhopper, rice stemfly, rice water weevil, white-backed planthopper, and yellow rice borer.
 - Wheat/Grain Pests: English grain aphid, wheat blossom midge.
 - Other Crop Pests: Army worm, corn borer, red spider, wireworm, large cutworm, and black cutworm.
 - General Categories: Various types of ants, bees, beetles, caterpillars, earthworms, earwigs, grasshoppers, moths, slugs, snails, and weevils.

The training data was converted from COCO format to a dataset compatible with the huggingface ecosystem. It was then pre-processed based on steps sourced from HuggingFace's official guide (Paniego, 2026) including:

- Filtering out invalid bounding boxes where the x_max or y_max values were smaller than the x_min or y_min values
- Data augmentation, involving artificially expanding the dataset
- Converting the bounding boxes from Pascal Voc to COCO format

Finally, the Peking model was trained on this data over three epochs for efficiency with a learning rate of 0.00001. The trained models were saved using Joblib for integration with the final application.

Intent Handling

This is the process of interpreting the user's input to identify its goal. The selection of the intent handling models replicated the study by Hassan et al (2022). As mentioned in the Design section, this project will use Scikit-learn's implementation of the SVM, K-NN, LR, MNB and RF algorithms. The dataset used for testing was the SPAM dataset sourced from UCI's ML repository.

Preprocessing

The SPAM dataset was imported from UCI's repository through the ucimlrepo library (UCI Machine Learning Repository, 2023). The classes are labelled using a binary format where 1=spam and 0=ham. As shown in Figure 19, the dataset is unbalanced (more ham examples than spam). This makes evaluation using precision, recall and f1-score crucial to balance out the dataset's skewed nature.

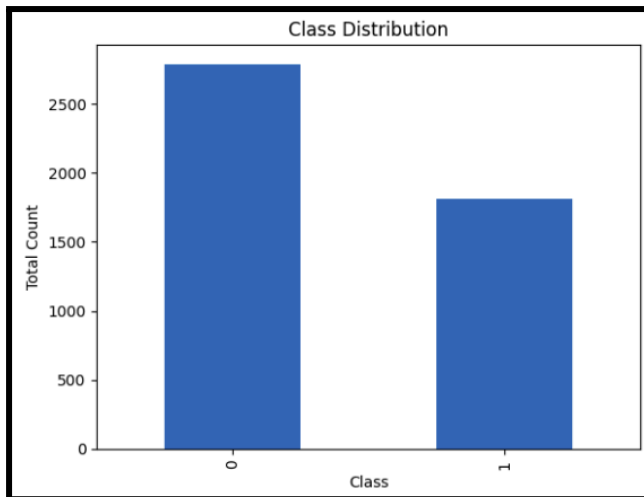


Figure 19: Spam label distribution

Testing

The dataset was first split into 80/20 train-test splits. Tuning was implemented using grid search cross-validation with five splits and f1_weighted scoring. The hyperparameters selected were:

- SVM:
 - Clf__c - a regularization parameter selecting either a low C of 0.1 for a smooth decision boundary or high C of 10 forcing the model to classify all training points correctly.
 - Clf__kernel - defines the shape of the decision boundary selecting between Linear and Radial Basis Function boundaries.
- K-NN:
 - Clf__n_neighbors: set 3, 5 and 11 as the number of neighbours to look at
 - Clf__weights: a uniform hyperparameter stating that each neighbor gets one vote or a distance hyperparameter stating that closer neighbors have more influence.
- LR:
 - Clf__c: similar to SVM
 - Clf__penalty: only one hyperparameter option, l2, set for this which discourages large coefficients
- MNB:
 - Clf__alpha: this is Laplace smoothing selecting between 0.1, 0.5 and 1.0 as the small count to add to each word.
- RF:
 - Clf__n_estimators: selects between 50 and 100 as the number of trees in the forest
 - Clf__max_depth: selects between None, 10 and 20 as the maximum height for the trees

The testing process is shown in Listing C.1, Appendix C.

Figure 20 shows the model's performance.

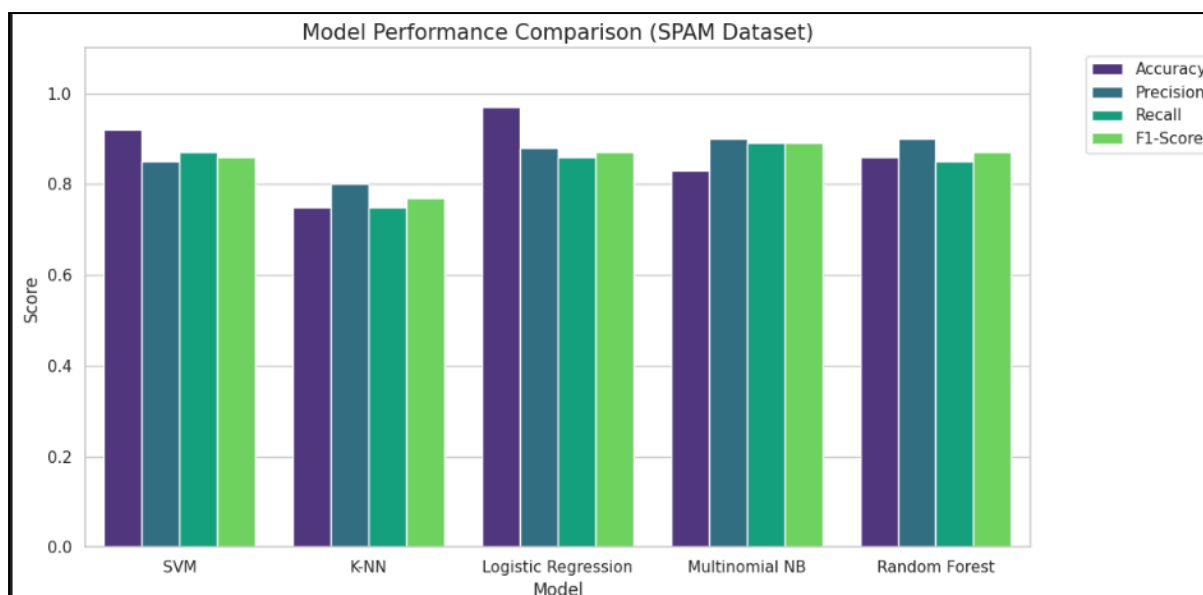


Figure 20 Model performance for SPAM dataset

According to the Design, the metric to be prioritized was F1-score. However, since four models (SVM, LR, MNB and RF) had relatively similar performances for this metric, accuracy was used instead. Unlike in the original paper, where SVM is the best performer on the SPAM dataset and LR has the lowest accuracy, this experiment found LR to still be the best performer. I will use the results from this experiment to select LR as the best model. However, future work may need to retest SVM and LR on different SPAM datasets to come to a better conclusion.

FineTuning

The LR model was fine-tuned on data sourced from an exploratory user survey. There was a 50-50 split between digitally literate and illiterate users to get an accurate representation of how the different demographics phrase their queries. Short interviews were done where participants were asked to give three to five questions they would ask a chatbot in these scenarios:

1. **Crop Simulation:** Imagine you were at the start of the planting season. What questions would you ask to help you prepare in terms of expected yield and water usage?
2. **Crop growth analysis:** It is the middle of the planting season and you suspect there is something wrong with your crops. How would you ask the chatbot for help?
3. **General conversation:** How would you start an interaction with the chatbot?

The responses were recorded and saved in a CSV file containing the text and intent. Due to the small sample size (13 individuals), the dataset collected was too small and required augmentation. This was done by duplicating text with common words replaced by their synonyms (Listing C.2, Appendix C). Finally, the augmented data was used to fine-tune the model which was then saved using Joblib.

Chatbot

The chatbot component consists of three main features:

- Text generation model
- Speech recognition model
- Speech synthesis model

Text Generation

The data obtained from the Chatbot Arena contained both the users' prompts, responses from two LLM models and the users' selection of the best model response. To test the text generation models, the response from the user's preferred model was used as the reference for each prompt. The BERTScore, WER and toxicity scores were then calculated using the reference and each model's generated response for each prompt. This process is shown in Listing D.1, Appendix D.

The testing results revealed that the ReaderLM-v2 (jinai) model outperformed the others across all metrics and was thus selected.

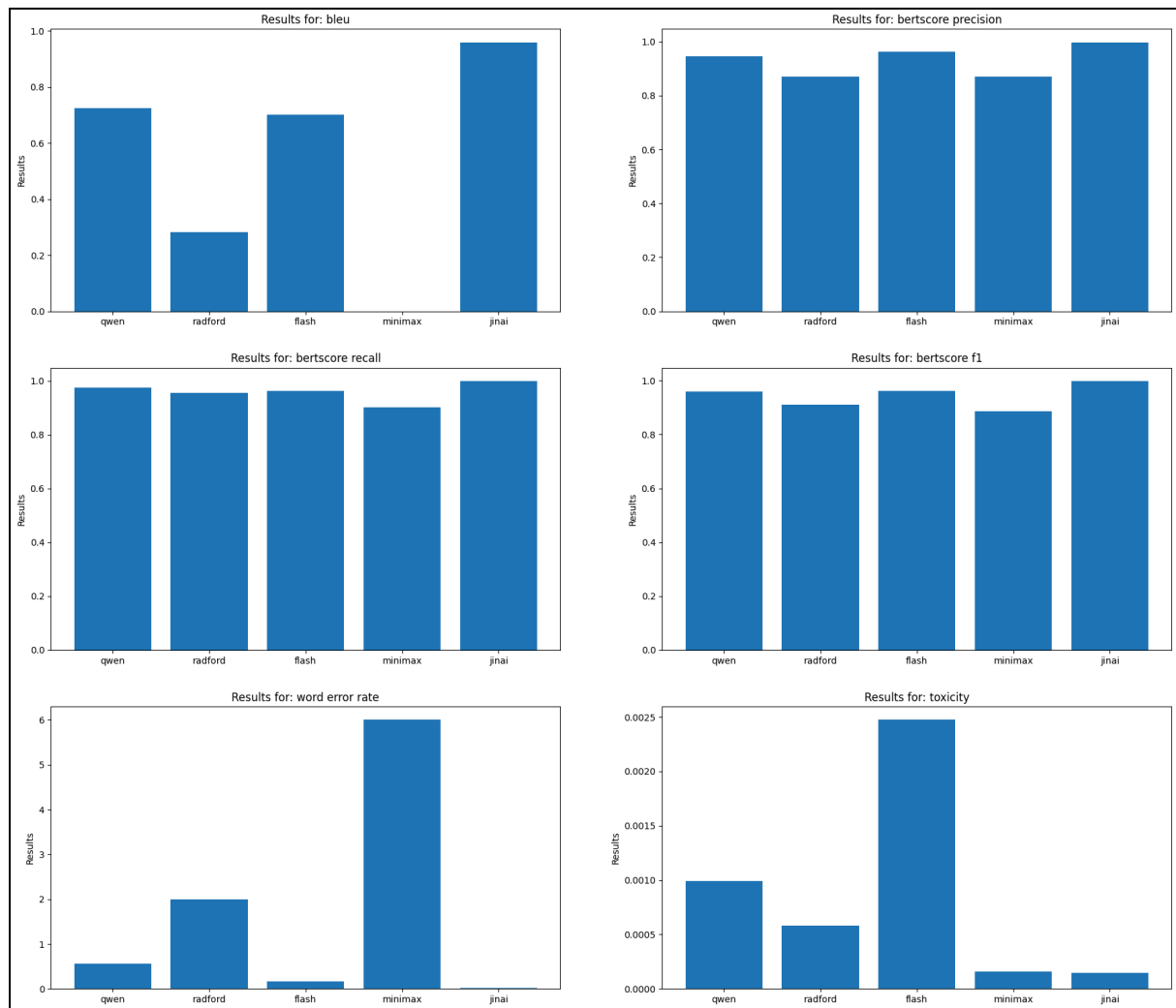


Figure 21: Text generation models performance

ReaderLM-v2's HuggingFace inference endpoint was used to integrate it into the application. This was because the model was too large to save in local storage and a smaller alternative

model was not found. Therefore, the application will need internet access for use impeding one of the aims of the project which was to develop a fully offline tool.

Speech Recognition

The models outlined in the Design section were tested using the AccentsDB dataset sourced directly from HuggingFace. Each model was supplied with an audio file input then the CER and toxicity metrics were evaluated based on the model's response and reference texts as shown in Listing E.1, Appendix E.

The results of the testing are visualised in Figure 22 showing that the Vosk model had the best performance.

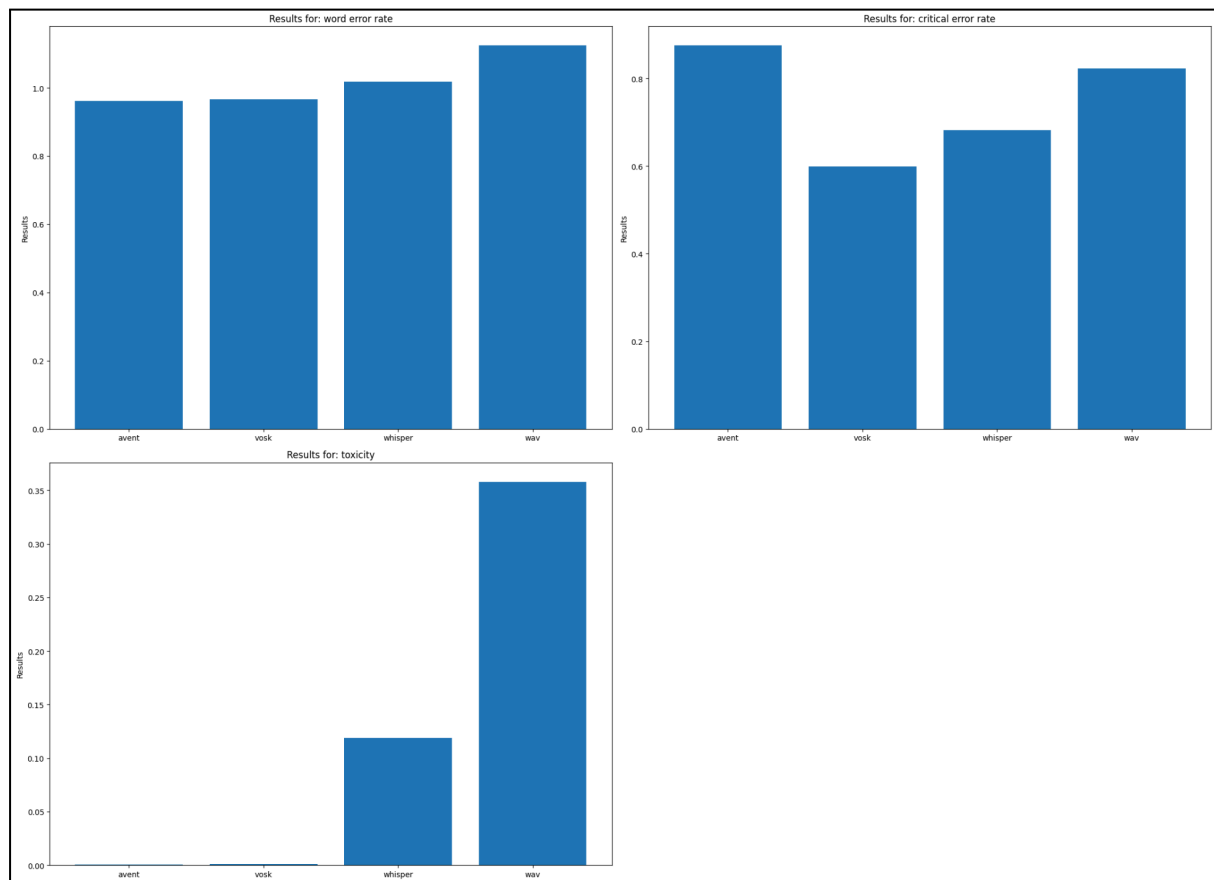


Figure 22: Results of testing the speech recognition models

The Vosk model was integrated into FarmersBot through the Vosk Python library.

Speech Synthesis

As planned in the Design section, real user feedback was used to select the best speech synthesis model. This was done through the webpage shown in Figure 23 where users were asked to enter a prompt then play the responses from each model (this process was done by me if users could not type). The participants then selected the model they deemed best.

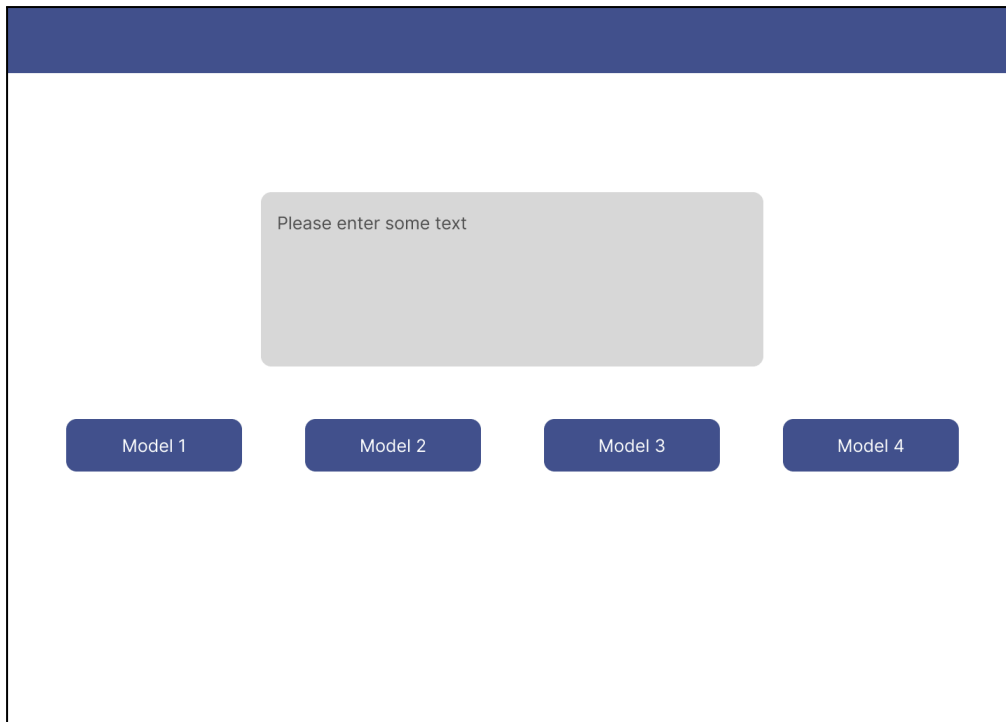
A screenshot of a web application interface. At the top is a dark blue header bar. Below it is a large, light gray rectangular text input area with the placeholder text "Please enter some text". At the bottom of the page, there are four dark blue buttons arranged horizontally, labeled "Model 1", "Model 2", "Model 3", and "Model 4" from left to right.

Figure 23: Webpage for testing the speech models

The results of this process, shown in Figure 24, found Kokoro to be best suited for this task.

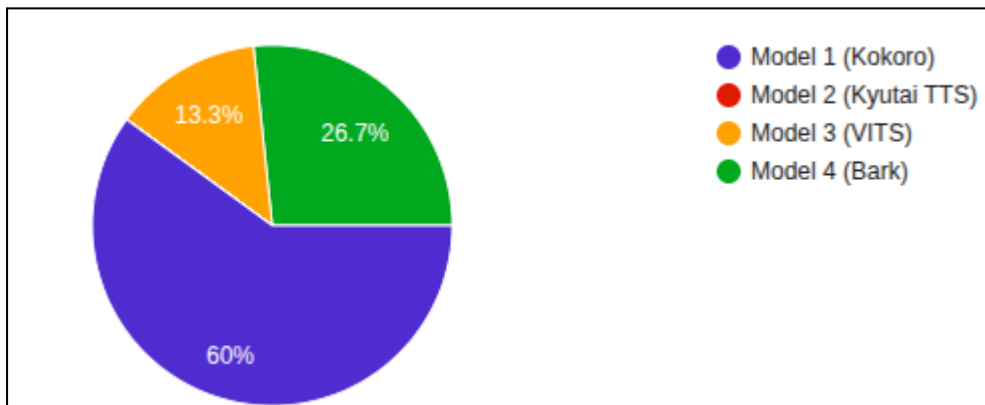


Figure 24: Results for speech synthesis models experiments

Originally, the Kokoro model was to be integrated using an inference endpoint but some packages required for this were not compatible with my computer and most smartphones. Therefore, a Gradio space (Muthoni, 2026) was created for the model so that it can be accessed through an API as shown in Listing F.1, Appendix F.

Server Coding

This phase involved development of the backend functions that control how data is transferred between the frontend and the various models. The main functions include:

- signup (Listing G.1, Appendix G) - creates a new user using a username, password, preferred interaction method and subcounty
- login (Listing G.2, Appendix G) - allows existing users to login using a username and password

- `handle_message` (Listing G.3, Appendix G) - receives user's text-based input and passes it to `handle_intent`
- `handle_image` (Listing G.4, Appendix G) - receives users' image input and passes it to `handle_intent`
- `handle_audio` (Listing G.5, Appendix G) - handles both the transfer of audio between the front and backend and audio-to-text (lines 34-38) and text-to-audio (lines 42-46) conversion
- `handle_intent` (Listing G.6, Appendix G) - passes user input to the intent model that predicts users' intent. Based on this prediction, `handle_intent` passes the text to the relevant function. Instances where additional information is required from the user (crop name for crop simulation and crop image for crop growth analysis) are also handled here as shown in lines 17-20 and 32-35.
- `get_simulation_data` (Listing G.7, Appendix G) - ensures that the required data for crop simulation is provided before simulation is done by the `run_simulation` function discussed earlier.
- `analyze_image` (Listing G.8, Appendix G) - receives the users' image and passes it through the disease and pest detection models.

The development language used was Python3.12 supported by the following libraries:

- FastAPI
- Uvicorn
- SQLAlchemy
- Passlib
- Alembic
- Joblib
- Numpy and Pandas
- Huggingface_hub, Vosk, OpenCV Python and Gradient Client

Frontend Development

The frontend implemented the signup/login page, a page for audio and text users and a settings page. The design was simplistic making it easily accessible to digitally illiterate individuals.

The signup and login pages are controlled by the CardTemplate widget (Listing H.1, Appendix H) where the distinction between them is that the interaction and subcounty selection elements are hidden for login as shown in lines 69-71 and 113-116.

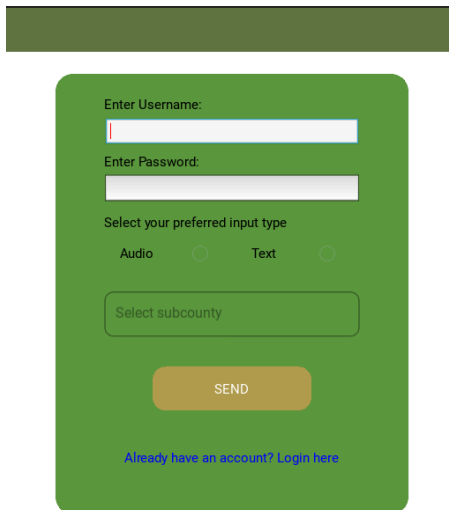


Figure 25 shows a signup page with a green background. At the top, there is a dark green header bar. The main content area contains the following elements: a label "Enter Username:" followed by a white text input field; a label "Enter Password:" followed by a white password input field; a label "Select your preferred input type" followed by two radio buttons, "Audio" (selected) and "Text"; a rounded rectangular button labeled "Select subcounty"; a large rounded rectangular orange button labeled "SEND"; and a link "Already have an account? Login here" at the bottom.

Figure 25: Signup page

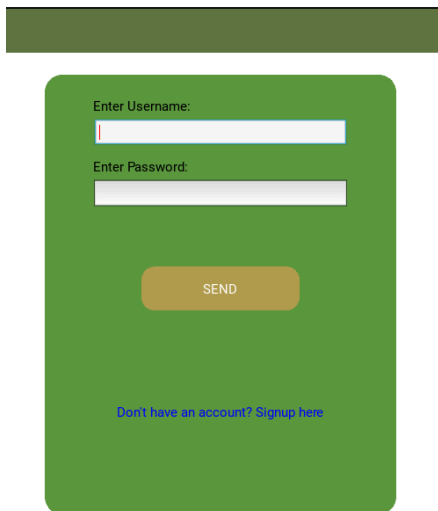


Figure 26 shows a login page with a green background. At the top, there is a dark green header bar. The main content area contains the following elements: a label "Enter Username:" followed by a white text input field; a label "Enter Password:" followed by a white password input field; a large rounded rectangular orange button labeled "SEND"; and a link "Don't have an account? Signup here" at the bottom.

Figure 26: Login page

The audio input page allows the user to click the large round button to begin audio recording and the smaller rectangular button to play the response. The latter button is disabled when the audio response has not been generated yet (Listing H.2, Appendix H).

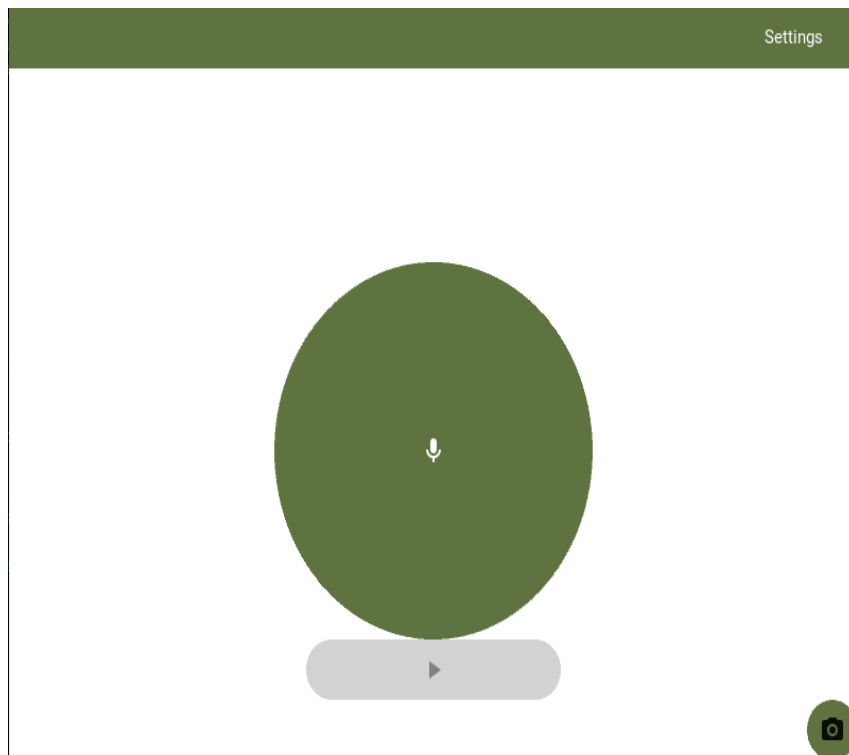


Figure 27: Audio interaction page

The settings page allows users to adjust all their details but was mainly set up to allow for switching between audio and text based interactions.

Figure 28: Settings page

The frontend was developed using Python3.12, with the main supporting library being Kivy, which handled the creation of smartphone elements. Additional libraries include:

- Pygame
- Sounddevice

The final source code can be found here: Source code:

<https://github.com/Hannah-Muthoni-Hailu/FarmersBot-AI-Assistant>

Evaluation (1829/2500 words)

Heuristic Evaluation

Heuristic evaluation shaped the evaluation process by allowing me to identify key issues in the interface to be prioritized during user testing (Moran and Gordon, 2023). While this evaluation is best performed in a team (Moran and Gordon, 2023), this project only used one evaluator due to the limited time scope inhibiting the creation of a team.

The Ten Commandments of Heuristics by Molich and Nielsen (1990) were used for the evaluation. The results of the heuristic research are documented below:

- **Heuristic:** Simple and natural dialogue
 - **Problem:** The icons used for the different buttons may be unfamiliar to digitally illiterate users.
 - **User impact:** Some users may need training to use the application which may be unavailable.
- **Heuristic:** Provide feedback
 - **Problem:** Some of the models take a long time to generate response. Since the frontend stays on the same page until a response is sent, the user may be left guessing on whether their prompt was received or not.
 - **User impact:** Users may be unsure about the state of their interaction as the system loads causing frustration as the application appears unresponsive.
- **Heuristic:** Provide clearly marked exits
 - **Problem:** Once the user has submitted a request, there is no way to stop the response generation.
 - **User impact:** The user has no way to interrupt response generation in cases where they may have submitted faulty input.
- **Heuristic:** Provide shortcuts
 - **Problem:** On the text input page, the user can only send input by pressing the send button.
 - **User impact:** Digitally literate users may tend to press the enter key to send input, since this is a common mental model (BriiMcfly, 2017). Being forced to use the send button may be frustrating for them.

Based on this evaluation, it is clear that the main issues are:

- Clarity of the input buttons and their functions to users
- General user-friendliness of the interfaces
- Timeliness of model responses
- Usability of the text input panel

Additional metrics to be evaluated that are not clearly defined by Molich & Nielsen's (1990) heuristics include accuracy, need for clarification and robustness of model responses.

User Testing - Model Performance

To evaluate model performance, a user study was conducted. While the automated testing techniques used during selection in the Implementation are effective for identifying high

performing models, integrating user feedback ensures that the final product is not only theoretically effective, but that it meets the needs of actual users (Trymata, 2023). Additionally, user feedback helps identify bugs missed during automated testing (Trymata, 2023).

Test Plan

The objectives were rating model performance in terms of:

- Accuracy - Are the responses correct and relevant to the prompt?
- Efficiency - Is using the model preferable to relying on traditional methods in relation to time used?
- Robustness - How well does the model handle unclear, incomplete, or edge-case inputs?

For each objective, the following tasks and success criteria were defined:

- Accuracy:
 - **Tasks**
 - Use the given model to perform [model specific task]
 - On a scale of 1 to 5, rate how accurate you found the response to be
 - On a scale of 1 to 5, rate how relevant you found the response to be
 - Do you feel the need to get clarification based on the model's response (Yes/No)
 - **Success criteria**
 - > 85% outputs are correct and relevant
 - > 90% of users do not require clarification
- Efficiency:
 - **Tasks**
 - Use the given model to perform [model specific task]
 - For the given task, do you think it would be more time effective to use the model or an alternative traditional method?
 - **Success criteria**
 - > 90% of users prefer the model to a traditional method
- Robustness:
 - **Tasks**
 - Input an unclear prompt into the model
 - On a scale of 1 to 5, how well do you think the model handled your query?
 - Input an edge-case prompt (for example, a crop that does not exist in the model's database) into the model
 - On a scale of 1 to 5, how satisfied are you with how the model handled the edge case
 - **Success criteria**
 - > 80% of users found that the model handled their edge case and unclear input in a satisfactory manner

User recruitment

A study group of 15 participants was obtained through opportunistic sampling from small-scale farmers in Kajiado county, Kenya. The participants were split into three categories:

- Literate users with no visual impairment - 8 participants
- Illiterate users with no visual impairment - 5 participants
- Users with visual impairment - 2 participants

Each of these categories would test a different set of models

- Literate users - Intent handling, crop growth simulation, crop growth analysis and general conversation models
- Illiterate users - Text and speech conversion models.
- Users with visual impairment - Text and speech conversion models.

Testing and Data Collection

Webpages were developed, such as the one in Figure 29, for each model that allowed the user to enter a prompt for the model and receive a response.

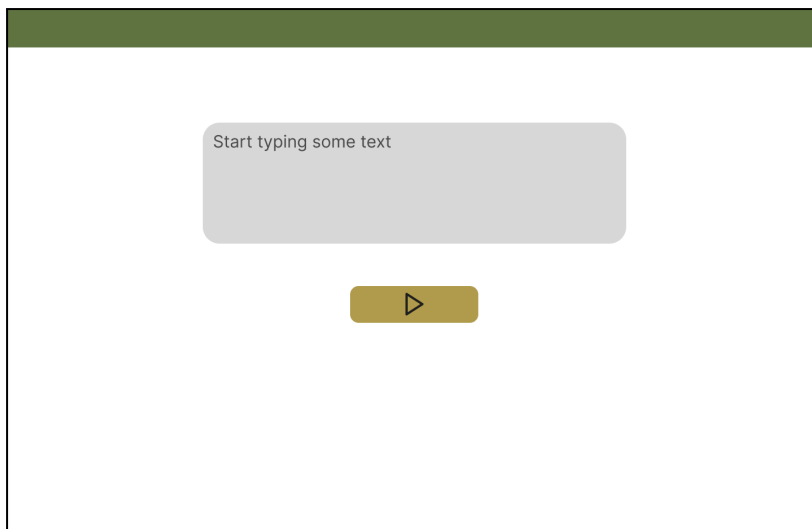


Figure 29 frontend design for text-to-audio testing webpage

The webpages were hosted on my computer and each participant was asked to interact with the list of models specified for their demographic, providing three different prompts for each task. They would then rank their experience as specified in the test plan. Table 1 outlines the results of this process.

Model Name	Objective	Task	Metric	Rating
Intent Handling	Accuracy	Input a prompt requesting crop simulation / crop growth analysis	Accuracy	4
			Relevance	3
			Need for clarification	Yes=10% No=90%
	Efficiency	Input a prompt requesting crop simulation / crop growth analysis	Efficiency	Model=100% Alternative=0%
	Robustness	Input an unclear prompt	Satisfaction	5
		Make a request for a non-sensical task	Satisfaction	4.5
Crop simulation	Accuracy	Request the model to simulate growth for a crop you'd like	Accuracy	5
			Relevance	5
			Need for clarification	Yes=0% No=100%
	Efficiency	Request the model to simulate growth for a crop you'd like	Efficiency	Model=100% Alternative=0%
	Robustness	Input an unclear prompt	Satisfaction	4
		Make a request for a simulation of a crop with the wrong spelling	Satisfaction	4
Crop growth analysis	Accuracy	You have been provided with a picture of a crop with a number of diseases and pests. Ask the model to identify these problems	Accuracy	5
			Relevance	5
			Need for clarification	Yes=40% No=60%
	Efficiency	You have been provided with a picture of a crop with a number of diseases and pests. Ask the model to identify these problems	Efficiency	Model=50% Alternative=50%
	Robustness	Input an unclear prompt	Satisfaction	3.5
		You have been provided with an image of an animal instead of a crop. Ask the model to identify problems in this crop	Satisfaction	2
General Conversation	Accuracy	Start a conversation with the bot like you would with a friend	Accuracy	3
			Relevance	2
			Need for clarification	Yes=70% No=30%
	Efficiency	Start a conversation with the bot like you would with a friend	Efficiency	Model=100% Alternative=0%
	Robustness	Input an unclear prompt	Satisfaction	3
		Start a non-sensical conversation with the bot	Satisfaction	3
Text to Speech	Accuracy	Provide any sentence to the moderator then listen to the audio that will be played	Accuracy	4.5
			Relevance	4.5
			Need for clarification	Yes=10% No=90%
	Efficiency	Provide any sentence to the moderator then listen to the audio that will be played	Efficiency	Model=100% Alternative=0%
	Robustness	Provide a sentence that you think may be difficult for someone who is not a native speaker of the language to understand	Satisfaction	5
		Provide a sentence with a word that has no actual meaning	Satisfaction	3.5
Speech to Text	Accuracy	Speak into the microphone (you are free to say anything) then listen to what the moderator reads out as the output from the model	Accuracy	3.5
			Relevance	3
			Need for clarification	Yes=71.4% No=28.6%
	Efficiency	Speak into the microphone (you are free to say anything) then listen to what the moderator reads out as the output from the model	Efficiency	Model=100% Alternative=0%
	Robustness	Provide a sentence that you think may be difficult for someone who is not a native speaker of the language to understand	Satisfaction	2
		Provide a sentence with a word that has no actual meaning in the specific language	Satisfaction	2.5

Table 1: Results for model performance testing

Analysis

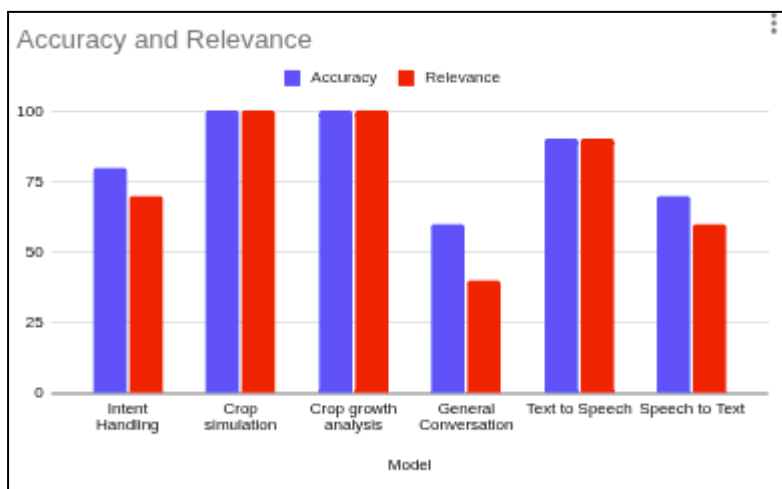


Figure 30: Model performance: Accuracy and Relevance

Figure 30 visualizes the models' performances in relation to accuracy. The only models that met the success criteria were the crop simulation (Accuracy = 100%, Relevance = 100%), crop growth analysis (Accuracy = 100%, Relevance = 100%) and text to speech (Accuracy =

90%, Relevance = 90%) models. The rest; intent handling (Accuracy = 80%, Relevance = 60%), general conversation (Accuracy = 60%, Relevance = 40%) and speech to text (Accuracy = 70%, Relevance = 70%) performed poorly. This indicates that poor fine-tuning data was used for the intent handling model and the selection process for all the models may have had flaws. Future work should therefore prioritize accuracy for model selection and use better fine-tuning data for the intent handling model.

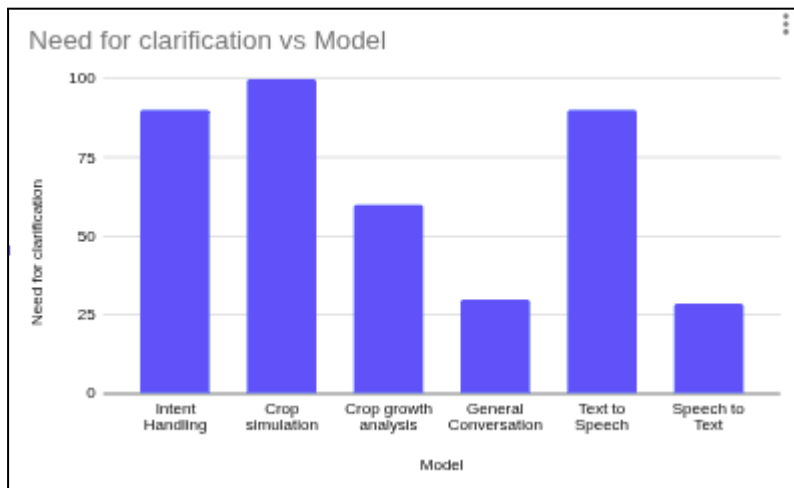


Figure 31: Model performance: Need for clarification

Only the intent handling, crop simulation and text to speech models met the success criteria in relation to need for clarification with scores of 90%, 100% and 90% respectively. This means that the remaining models offered unclear responses. This could be for the speech to text model through better model selection strategies. As for the crop growth analysis model, a solution could involve testing out different ways to convert the model's responses that are in dictionary form to prose user-readable prose form.

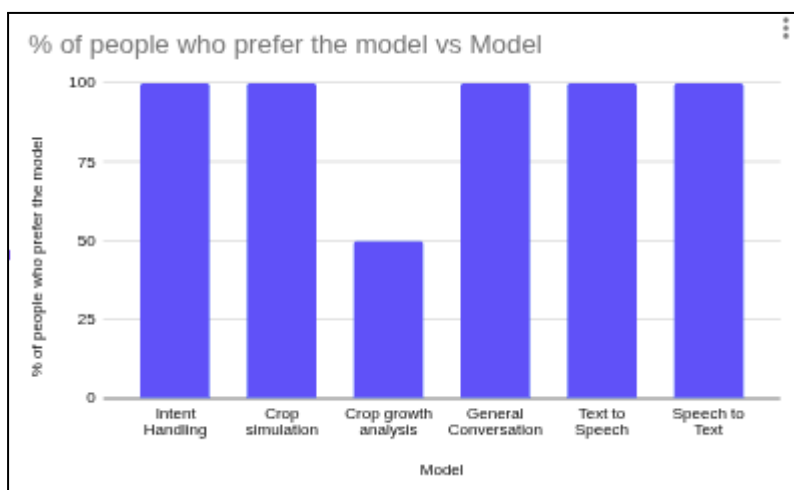


Figure 32: Model performance: Efficiency

All the models met the success criteria for efficiency except the crop growth analysis model where 50% of participants stated that they preferred a traditional approach (an agronomist). This may indicate a lack of trust in this model specifically which needs to be addressed in future versions.

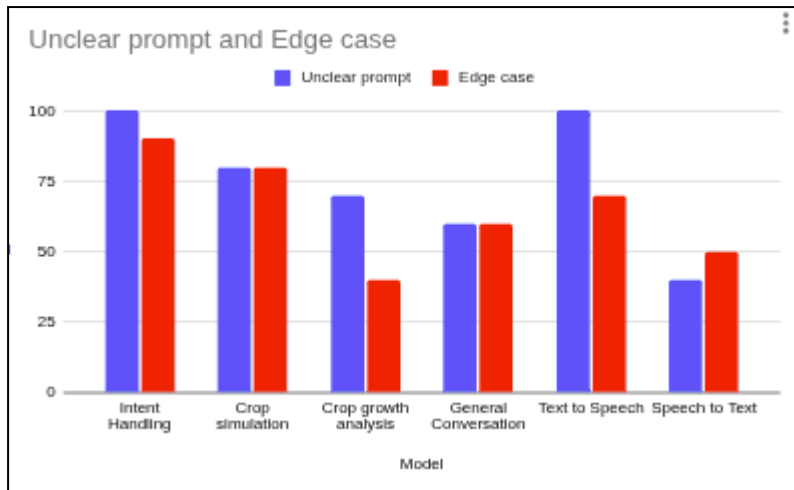


Figure 33: Model performance: Robustness

In relation to robustness, only the intent handling and crop simulation models were able to meet the success criteria. The text to speech model was able to handle unclear prompts with a 100% score but significantly underperformed when it came to edge cases with a score of 70%. The remaining models performed poorly indicating a general failing with the application in relation to robustness. Future versions should focus on testing out the models' ability to handle edge cases and unclear prompts as part of the model selection process.

User Testing - Application

User testing was used to evaluate the effectiveness and user-friendliness of the application. This allowed for iterative improvements to be made to the application before widespread deployment as detailed in this section.

First Iteration

The first iteration is as shown in the Frontend Development section of the Implementation.

Testing

This iteration was presented to a study group consisting of 25 participants. For this small scale testing, the application was deployed as an Android APK. This had the limitation that it could not be tested by iPhone users. The participants were directed on how to install the APK on their devices and how to create an account and use the basic features of the application. They were then given a period of 3 days to test out the application by themselves.

After the test period, a System Usability Scale (SUS) study was conducted to obtain qualitative data that would help determine the user-friendliness of the application (Soegaard, 2025). The users were split into two groups:

- Literate users - 15 participants who had been directed to use text-based interaction
- Illiterate users/users with visual impairments - 10 participants who had been directed to use audio-based interaction

The group of literate users was presented with a printed version of the questionnaire in Figure 34. Closed-ended interviews were conducted with the other group using similar questions to those in the questionnaire.

FarmersBot

Please fill out this questionnaire based on your experience using the FarmersBot application for the past three days.

muthonihannahhailu@gmail.com [Switch accounts](#)

Not shared

* Indicates required question

I think that I would like to use this system frequently. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I found the system unnecessarily complex. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I thought the system was easy to use. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I think that I would need the support of a technical person to be able to use this system. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I found the various functions in this system were well integrated. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I thought there was too much inconsistency in this system. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I would imagine that most people would learn to use this system very quickly. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I found the system very cumbersome to use. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I felt very confident using the system. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

I needed to learn a lot of things before I could get going with this system. *

1 2 3 4 5

Strongly Disagree ☐ ☐ ☐ ☐ ☐ Strongly Agree

Submit Clear form

Figure 34: Survey questions

The results are shown in Table 2.

Question	Questionnaire Result	Interview Result
I think that I would like to use this system frequently.	4	4.5
I found the system unnecessarily complex.	1	2
I thought the system was easy to use.	5	3
I think that I would need the support of a technical person to be able to use this system.	1	3.5
I found the various functions in this system were well integrated.	4	4.5
I thought there was too much inconsistency in this system.	1	1.5
I would imagine that most people would learn to use this system very quickly.	4	2.5
I found the system very cumbersome to use.	1	1.5
I felt very confident using the system.	4.5	3
I needed to learn a lot of things before I could get going with this system.	1.5	5
Total	90	60

Table 2: Survey results first iteration

The SUS score for text-based users was 90 which can be considered excellent. However, the audio based scores were significantly worse at 60, meaning that this should be the main focus for improvement in the second iteration. To determine what the exact challenges audio-based users faced were, an open-ended interview was conducted with the same participants where each was asked to describe what the main challenges they faced were.

The key points raised were:

- Confusion as to which button performed which task in relation to audio input and output
- Users were expected to login through text based interaction every time they opened the app making it impossible for audio-based users to login without assistance.

Based on this feedback, the following changes were implemented:

- The user would only have to click one button to start recording and audio playback would begin automatically after a response is generated.



Figure 35: Audio page with no audio playback button

- Allow users to remain logged into the application by saving their authentication token in local storage.

```

248         if audio_response.status_code == 200:
249             response_path = self._build_response_audio_path()
250             with open(response_path, "wb") as f:
251                 f.write(audio_response.content)
252             self.response_audio_path = response_path
253             threading.Thread(target=self.play_response, daemon=True).start()

```

Figure 36: Line 253 allows the audio to be played automatically after it is received from the backend

Second iteration

After the SUS study from the first iteration, it was determined that the main changes required were on the audio interaction page. First, audio playback after a response was generated was set to be automatic thus removing the need for two different buttons for audio recording and playback. Next, the login function was modified to save an authentication token in local storage for each user to remove the need for them to login every time.

Testing

Since the changes were mostly on the audio page, the second SUS survey was only conducted for audio-based users with the same questions as shown during the first iteration. The results of this second survey are shown in Table 3.

Question	Interview Result
I think that I would like to use this system frequently.	4.5
I found the system unnecessarily complex.	1.5
I thought the system was easy to use.	4
I think that I would need the support of a technical person to be able to use this system.	2
I found the various functions in this system were well integrated.	5
I thought there was too much inconsistency in this system.	1
I would imagine that most people would learn to use this system very quickly.	1.5
I found the system very cumbersome to use.	1
I felt very confident using the system.	4
I needed to learn a lot of things before I could get going with this system.	2
Total	78.75

Table 3: Survey results second iteration

The changes boosted the score up to a 78.75, which can be considered good but there is clearly room for improvement. Future versions of the application should re-evaluate the system with audio-based users, identifying further pain points.

Evaluation Based on Objectives

For the most part, the application met its goals, but there are still areas left for improvement. The main change hearkens back to the objectives set in the Introduction section. FarmersBot was supposed to have offline support which was not implemented since a number of models had to be deployed on remote servers and accessed through APIs or inference endpoints thus needing internet connectivity. However, all other objectives were met, that being the development of a user friendly application, as evidenced by the SUS scores, that offers crop simulation and crop growth analysis tools in the form of a simplistic chatbot.

Conclusion (899/1000 words)

This project aimed to develop a farmers' assistant called FarmersBot that leveraged AI models to provide:

- Crop simulation
- Crop growth analysis
- Multilingual audio and text based chatbot
- Offline access

The target demographic was small-scale farmers in rural parts of Kenya, who oftentimes lack access to digital agricultural tools due to illiteracy, lack of English fluency or limited access to the internet. This forces them to rely on traditional tools that are often faulty and result in heavy losses. Since agriculture can be considered the backbone of Kenya's economy, improving yield for local small-scale farmers not only benefits them, but also the country as a whole. This shows that there is a great need for a tool like FarmersBot.

During the time frame of this project, a working prototype of the application was developed with support for crop simulation, crop growth analysis and text and audio-based interactions. Through SUS surveys, it was determined that the application was user-friendly with a score of 90 among text-based users and 78.75 among audio-based users. This shows that the simplistic interface used for the application was easy to use and accessible for a broad demographic. Iterative improvements made during the user testing also helped to reduce ambiguity around the functions of certain features and address login challenges amongst illiterate users.

The crop simulation model performed extremely well during user testing in relation to accuracy, relevance, robustness and efficiency. This indicates that the selected model was effective at its task and that little improvement is needed for this feature. It could also point to a lack of such tools in the market and thus the novelty of such a tool was extremely appealing to most users. This is evidenced by the fact that 100% of the participants stated that they would rather use the tool than traditional alternatives.

In contrast, the crop growth analysis tool severely underperformed on all metrics other than accuracy. An important thing to note on its performance is in relation to efficiency where half of the participants stated that they would prefer a traditional approach, in this case an agronomist, over the model. This points to a lack of trust in the model among users probably due to its inability to properly handle edge cases or the need for clarification with the model's responses. Future iterations of FarmersBot should mostly focus on improving the crop growth analysis model in relation to edge-case handling and clarity of responses. Further surveys would also help to shed light on why there was so much distrust in the crop growth analysis model as compared to the simulation model.

Aside from the main crop simulation and growth analysis models, additional models were implemented including ones for intent handling, speech understanding and synthesis and general conversation. On average, these models failed to meet the expected criteria during user evaluation. Of significant note is the speech to text model which underperformed on all metrics other than efficiency. Its performance could signal an issue with the model selection

process, probably with regards to the dataset used or the choice of models tested. Future work should increase the number of models tested and source testing data from more reputable sources, probably even through a study similar to the one performed for the speech generation models. The intent handling model mostly fell short with regards to relevance signaling a potential need for better or more fine-tuning data. As for the speech understanding model, its performance was mostly mediocre with significant challenges noted for edge-case handling. Fine-tuning the model using hyperparameters could help address this.

While the project was able to meet most of its objectives, two significant ones were not meant. First, the application was intended to support rural Kenyan farmers with limited English fluency through machine translation of the output. However, due to time constraints, the machine translation models were not tested or implemented into the final product. This creates a barrier for entry for a large portion of the target user-base. Secondly, a number of the models had to be implemented remotely and accessed using API calls due to their large size. While the contingency plan stated that an alternative smaller model should be used in place of such models, the selection process of such a replacement was unfeasible within the time-frame given. Therefore, the application is only accessible online, which directly impacts the offline access objective specified at the start of the project.

This project was mostly successful in meeting its intended aim. Nonetheless, there still remains huge gaps for improvement that can be addressed in future work including:

- Implementation of a machine translation feature that allows the bot to interact in all the languages spoken in Kenya with room and other international languages.
- Using hyperparameter tuning to improve the crop growth analysis, intent handling, speech synthesis and general conversation models.
- Conducting research into the causes of distrust among users towards the crop growth analysis models and how this can be mitigated.
- Implementation of smaller alternative models in place of the larger ones that require remote access thus allowing the application to be usable offline.

Ultimately, this project shows that it is possible to bridge the gap between small-scale rural farmers in developing countries and access to digital tools. With further refinement, the system has the potential to meaningfully contribute to a digital transformation in agriculture especially in Kenya and the rest of the global south.

Reference List

AGRIVI. (2024). AGRIVI: Leading Farm Management Software for Digital Agriculture. [online] Available at: <https://www.agrivi.com/>.

ai.meta.com. (n.d.). Wav2vec 2.0: Learning the structure of speech from raw audio. [online] Available at: <https://ai.meta.com/blog/wav2vec-20-learning-the-structure-of-speech-from-raw-audio/>.

Ambali, O. I., Areal, F. J., & Georgantzis, N. (2021). Improved Rice Technology Adoption: The Role of Spatially-Dependent Risk Preference. *Agriculture*, 11(8), 691. <https://doi.org/10.3390/agriculture11080691>.

Arnon, D.I. & Stout, P.R. (1939). THE ESSENTIALITY OF CERTAIN ELEMENTS IN MINUTE QUANTITY FOR PLANTS WITH SPECIAL REFERENCE TO COPPER. *Plant Physiology*, 14(2), pp.371–375. doi:<https://doi.org/10.1104/pp.14.2.371>.

APSIM. (n.d.). Home. [online] Available at: <https://www.apsim.info/>.

Barbedo, J.G.A. (2019). Detection of nutrition deficiencies in plants using proximal images and machine learning: A review. *Computers and Electronics in Agriculture*, 162, pp.482–492. doi:<https://doi.org/10.1016/j.compag.2019.04.035>.

Bhatia, A. (2014). Crop Growth Simulation Modeling. *Modelling and Simulation of Diffusive Processes. Simulation Foundations, Methods and Applications*. Springer, Cham. https://doi.org/10.1007/978-3-319-05657-9_15.

Bianchi, F. (2025). Coolors. [online] Coolors.co. Available at: <https://coolors.co/>.

Bo, L., Liu Z., Huang J., Zhang L., Wan, Z. & Shi J. (2009). Hyperspectral identification of rice diseases and pests based on principal component analysis and probabilistic neural network. 25(9), pp.143–147. doi:<https://doi.org/10.3969/j.issn.1002-6819.2009.9.026>.

Bouma, D. (1983). Diagnosis of Mineral Deficiencies Using Plant Tests. In: Läubli, A., Bielecki, R.L. (eds) *Inorganic Plant Nutrition. Encyclopedia of Plant Physiology*, vol 15. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-68885-0_5.

BriiMcfly (2017). Form navigation mental model. [online] StackExchange. Available at:<https://ux.stackexchange.com/a/107570>.

Buhari, B. S., Sani, A. M., Umar, Z. & Malami I. M. (2023). A Historical Analysis and Pedagogical Implications of Artificial Intelligence and Chatbots. *Proc. of the International Conference on Computing and Advances in Information Technology*.

Canuma, P. (2026). accentsDB-with-transcripts. [online] Huggingface.co. Available at: <https://huggingface.co/datasets/prince-canuma/accentDB-with-transcripts>.

Cao, Q., Yang, G., Duan, D., Chen, L., Wang, F., Xu, B., Zhao, C. & Niu, F. (2022). Combining multispectral and hyperspectral data to estimate nitrogen status of tea plants (*Camellia sinensis* (L.) O. Kuntze) under field conditions. *Computers and Electronics in Agriculture*, 198, pp.107084–107084. doi:<https://doi.org/10.1016/j.compag.2022.107084>

Centralbank.go.ke. (2024). Agriculture Sector Survey of January 2024 | CBK. [online] Available at: <https://www.centralbank.go.ke/2024/02/09/agriculture-sector-survey-of-january-2024/>.

Chandel, A. (2025). Healthcare Chatbot Using SVM & Decision Tree. *Trends in Health Informatics*, 2(1), 10-17. <https://doi.org/10.22105/thi.v2i1.26>.

Chen, L., Lin, L., Cai, G., Sun, Y., Huang, T., Wang, K. & Deng, J. (2014). Identification of Nitrogen, Phosphorus, and Potassium Deficiencies in Rice Based on Static Scanning Technology and Hierarchical Identification Method. *PLoS ONE*, [online] 9(11). doi:<https://doi.org/10.1371/journal.pone.0113200>.

Cheng, S., Bao, Y., Cao, Q., Huang, L., Kang, L., Liu, Z., ... Wu, Y. (2025). Seed-X: Building Strong Multilingual Translation LLM with 7B Parameters. *arXiv [Cs.CL]*. Retrieved from <http://arxiv.org/abs/2507.13618>

Choi, D., & Lee, S. (2021). Comparison of machine learning algorithms for predicting lane changing intent. *International journal of automotive technology*, 22(2), 507-518.

Climate.com. (2025). Maximize Results with Our Digital Farming Solution | Climate FieldView. [online] Available at: <https://climate.com/en-us.html>.

Coggins, S., McCampbell, M., Sharma, A., Sharma, R., Haefele, S.M., Karki, E., Hetherington, J., Smith, J. and Brown, B. (2022). How have smallholder farmers used digital extension tools? Developer and user voices from Sub-Saharan Africa, South Asia and Southeast Asia. *Global Food Security*, [online] 32, p.100577. doi:<https://doi.org/10.1016/j.gfs.2021.100577>.

commonvoice.mozilla.org. (n.d.). Common Voice by Mozilla. [online] Available at: <https://commonvoice.mozilla.org/en>.

Cunha, M.L.P., Aquino, L.A., Novais, R.F., Clemente, J.M., de A, P.M. & Oliveira, T.F. (2016). Diagnosis of the Nutritional Status of Garlic Crops. *Revista Brasileira de Ciência do Solo*, 40(0). doi:<https://doi.org/10.1590/18069657rbcS20140771>.

Cvedia.com. (2026). COCOEval - CVEDIA-RT. [online] Available at: <https://docs.cvedia.com/develop/plugins/processing/cocoeval/index.html>.

Dimkpa, C., Bindraban, P., McLean, J.E., Gatere, L., Singh, U. & Hellums, D. (2017). Methods for Rapid Testing of Plant and Soil Nutrients. In: Lichtfouse, E. (eds) Sustainable Agriculture Reviews. Sustainable Agriculture Reviews, vol 25. Springer, Cham.
https://doi.org/10.1007/978-3-319-58679-3_1.

Dowding, J., Gawron, J. M., Appelt, D., Bear, J., Cherny, L., Moore, R. C., & Moran, D. B. (1993). Gemini: A natural language system for spoken-language understanding. In 31st Annual meeting of the Association for Computational Linguistics (pp. 54-61).

DSSAT.net. (n.d.). DSSAT.net. [online] Available at: <https://dssat.net/>.

Fang, Y., Liao, B., Wang, X., Fang, J., Qi, J., Wu, R., Niu, J., & Liu, W. (2021). You Only Look at One Sequence: Rethinking Transformer in Vision through Object Detection. CoRR, abs/2106.00666. <https://doi.org/10.48550/arXiv.2106.00666>

gantian127 (2025). soilgrids/notebooks/soilgrids.ipynb at master · gantian127/soilgrids. [online] GitHub. Available at: <https://github.com/gantian127/soilgrids/blob/master/notebooks/soilgrids.ipynb>.

Hamroun, M., & Gouider, M. S. (2020). A survey on intention analysis: successful approaches and open challenges. Journal of Intelligent Information Systems, 55(3), 423-443.

Hassan, S. U., Ahamed, J., & Ahmad, K. (2022). Analytics of machine learning-based algorithms for text classification. Sustainable operations and computers, 3, 238-248.

He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask r-cnn. In Proceedings of the IEEE international conference on computer vision (pp. 2961-2969).

Hefny, A. H., Dafoulas, G. A., & Ismail, M. A. (2020). Intent classification for a management conversational assistant. In 2020 15th International Conference on Computer Engineering and Systems (ICCES) (pp. 1-6). IEEE.

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). Spambase [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C53G6X>.

Hu, Y., Deng, X., Lan, Y., Chen, X., Long, Y., & Liu, C. (2023). Detection of Rice Pests Based on Self-Attention Mechanism and Multi-Scale Feature Fusion. Insects, 14(3), 280.
<https://doi.org/10.3390/insects14030280>.

Huggingface.co. (2022). facebook/nllb-200-distilled-600M · Hugging Face. [online] Available at: <https://huggingface.co/facebook/nllb-200-distilled-600M>.

Huggingface.co. (2024). hexgrad/Kokoro-82M · Hugging Face. [online] Available at: <https://huggingface.co/hexgrad/Kokoro-82M>.

Huggingface.co. (2023). suno/bark-small · Hugging Face. [online] Available at: <https://huggingface.co/suno/bark-small>.

Huggingface.co. (2026). AventureAI/whisper-audio-to-text · Hugging Face. [online] Available at: <https://huggingface.co/AventureAI/whisper-audio-to-text>

Huggingface.co. (2026). MiniMaxAI/MiniMax-M2.1 · Hugging Face. [online] Available at: <https://huggingface.co/MiniMaxAI/MiniMax-M2.1>

ict.go.ke. (2017). Last Mile County Connectivity Project (LMCCP) | Ministry of ICT and the Digital Economy. [online] Available at: <https://ict.go.ke/last-mile-county-connectivity-project-lmccp>.

Imdb.com. (2023). IMDb Non-Commercial Datasets. [online] Available at: <https://developer.imdb.com/non-commercial-datasets/>.

Ingram, K.T. & McCloud, D.E. (1984). Simulation of Potato Crop Growth and Development 1. Crop Science, 24(1), pp.21–27.
doi:<https://doi.org/10.2135/cropsci1984.0011183x002400010006x>.

Isric.org. (2025). Soilgrids REST API - Swagger UI. [online] Available at: <https://rest.isric.org/soilgrids/v2.0/docs>.

Jabed, M. A., & Azmi M., (2024). Crop yield prediction in agriculture: A comprehensive review of machine learning and deep learning approaches, with insights for future research and sustainability. Heliyon, 10(24), e40836.
<https://doi.org/10.1016/j.heliyon.2024.e40836>.

Jame, Y.W. & Cutforth, H.W. (1996). Crop growth models for decision support systems. Canadian Journal of Plant Science, 76(1), pp.9–19.
doi:<https://doi.org/10.4141/cjps96-003>.

Jiang, A. Q., Sablayrolles, A., Mensch, A., & Others. (2023). Mistral 7B. arXiv [Cs.CL]. Retrieved from <http://arxiv.org/abs/2310.06825>

Jina AI. (2025). ReaderLM v2: Frontier Small Language Model for HTML to Markdown and JSON. [online] @JinaAI_. Available at: <https://jina.ai/news/readerlm-v2-frontier-small-language-model-for-html-to-markdown-and-json/>

Juma, M., Nyangena, W. & Yesuf, M., (2022). Production risk and farm technology adoption in rain-fed, semi-arid lands of Kenya. Environment for Development Initiative.

Kalaji, H.M., Oukarroum, A., Alexandrov, V., Kouzmanova, M., Brestic, M., Zivcak, M., Samborska, I.A., Cetner, M.D., Allakhverdiev, S.I. & Goltsev, V. (2014). Identification of

nutrient deficiency in maize and tomato plants by in vivo chlorophyll fluorescence measurements. *Plant Physiology and Biochemistry*, 81, pp.16–25.
doi:<https://doi.org/10.1016/j.plaphy.2014.03.029>.

Kanani, K. C., & Sanghani, G. B. (2025). The Evolution of Intent Detection: Deep Learning-Based Paradigms for Natural Language Understanding. *Cureus Journal of Computer Science*.

Kilimo.go.ke. (2019). KilimoSTAT. [online] Available at:
<https://statistics.kilimo.go.ke/sw/KenyafarmingHH/>.

KNBS (2024). NATIONAL AGRICULTURE PRODUCTION REPORT. [online] Available at:
<https://www.knbs.or.ke/wp-content/uploads/2025/01/National-Agriculture-Production-Report-2024.pdf>.

Krell, N. T., Giroux, S. A., Guido, Z., Hannah, C., Lopus, S. E., Caylor, K. K., & Evans, T. P. (2021). Smallholder farmers' use of mobile phone services in central Kenya. *Climate and Development*, 13(3), 215-227.

Lazar, J. (2023). A Framework for Born-Accessible Development of Software and Digital Content. In: Abdelnour Nocera, J., Kristín Lárusdóttir, M., Petrie, H., Piccinno, A., Winckler, M. (eds) *Human-Computer Interaction – INTERACT 2023*. INTERACT 2023. Lecture Notes in Computer Science, vol 14145. Springer, Cham.
https://doi.org/10.1007/978-3-031-42293-5_32.

Lin, C.-C., Huang, A.Y.Q. & Yang, S.J.H. (2023) 'A Review of AI-Driven Conversational Chatbots Implementation Methodologies and Challenges (1999–2022)', *Sustainability*, 15(5), available:
<http://dx.doi.org/10.3390/su15054012>

Lin, T., Wang, Y., Liu, X. & Qiu, X. (2022). A survey of transformers. *AI Open*, 3, pp.111–132.
doi:<https://doi.org/10.1016/j.aiopen.2022.10.001>.

Lin, T. Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2117-2125).

Lin, T.-Y., Maire, M., Belongie, S., Lubomir Bourdev, Girshick, R., Hays, J., Perona, P., Ramanan, D., C. Lawrence Zitnick and Piotr Dollár (2014). Microsoft COCO: Common Objects in Context. *arXiv* (Cornell University). doi:<https://doi.org/10.48550/arxiv.1405.0312>.

Lingyan X., Ma, B., Zhu, K. & Zhang, L. (2018). Implementation and optimization of image acquisition with smartphones in computer vision. *2022 International Conference on Information Networking (ICOIN)*, pp.261–266.
doi:<https://doi.org/10.1109/icoin.2018.8343121>.

Liu, J., Li, Y., & Lin, M. (2019). Review of intent detection methods in the human-machine dialogue system. In *Journal of physics: conference series* (Vol. 1267, No. 1, p. 012059). IOP Publishing.

Lmarena.ai. (2024). Chatbot Arena (formerly LMSYS): Free AI Chat to Compare & Test Best AI Chatbots. [online] Available at: <https://lmarena.ai/>.

Lone, A. F., Kumar, Y. R., Kumar, S., Rasool, F. U., Abidi, I. A., Kumar, A., Lone, B., Dar, Z. & Nisar, F. (2021) Crop Simulation Models: A Tool for Future Agricultural Research and Climate Change. *Asian Journal of Agricultural Extension, Economics & Sociology*. 146-154. 10.9734/ajaees/2021/v39i630602.

Lu, J., Tan, L., & Jiang, H. (2021). Review on Convolutional Neural Network (CNN) Applied to Plant Leaf Disease Classification. *Agriculture*, 11(8), 707. <https://doi.org/10.3390/agriculture11080707>.

Luciel R. D., Angelo, L., Batista, F., Clemente, J.M. & Natália S. A. (2016). Diagnostic Methods to Assess the Nutritional Status of the Carrot Crop. *Revista Brasileira de Ciência do Solo*, 40(0). doi:<https://doi.org/10.1590/18069657rbcS20140813>.

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 142–150. Retrieved from <http://www.aclweb.org/anthology/P11-1015>

Michieka, M. M. (2011). A Second or Foreign Language? Unveiling the Realities of English in Rural Kisii, Kenya. Pp.206–216.

Mohanty, S.P., Hughes, D.P. & Marcel, S. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Front. Plant Sci.* 7, 1419.

Molich, R. and Nielsen, J. (1990). Improving a human-computer dialogue. *Communications of the ACM*.

Moran, K. and Gordon, K. (2023). How to Conduct a Heuristic Evaluation. [online] Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/>.

Muñoz-Huerta, R. F., Guevara-Gonzalez, R. G., Contreras-Medina, L. M., Torres-Pacheco, I., Prado-Olivarez, J., & Ocampo-Velazquez, R. V. (2013). A Review of Methods for Sensing the Nitrogen Status in Plants: Advantages, Disadvantages and Recent Advances. *Sensors*, 13(8), 10823-10843. <https://doi.org/10.3390/s130810823>.

Muthoni, H. (2026). Kokoro Audio. [online] Huggingface.co. Available at: <https://huggingface.co/spaces/Muthoni254/kokoro-audio>.

Nikitha, S., Prabhanjan, S. & Sathyanarayan. (2025). A. Plant nutritional deficiency detection: a survey of predictive analytics approaches. *Iran J Comput Sci* 8, 83–101.

<https://doi.org/10.1007/s42044-024-00209-5>

O Sullivan, K. (2018). Comparing the effectiveness of support vector machines and convolutional neural networks for determining user intent in conversational agents.

Obudho, G.M. (2025). National Agriculture Production Report. [online] Kenya National Bureau of Statistics, Real Towers, Hospital Road, Upper Hill: Kenya National Bureau of Statistics , pp.xii–xiv. Available at:
<https://www.knbs.or.ke/reports/national-agriculture-production-report-2025/>.

OpenAI (2022). Introducing Whisper. [online] Openai.com. Available at:
<https://openai.com/index/whisper/>.

Pps.wur.nl. (1989). LINTUL-1, a simple crop growth model to simulate growth under potential growing conditions | Models & Data Library. [online] Available at:
<https://models.pps.wur.nl/lintul-1-simple-crop-growth-model-simulate-growth-under-potential-growing-conditions>.

Paniego, S. (2026). Fine-Tuning Object Detection Model on a Custom Dataset, Deployment in Spaces, and Gradio API Integration. [online] Hugging Face. Available at:
https://huggingface.co/learn/cookbook/en/fine_tuning_detr_custom_dataset.

Pawlik, Ł., Płaza, M., Deniziak, S., & Boksa, E. (2022). A method for improving bot effectiveness by recognising implicit customer intent in contact centre conversations. *Speech Communication*, 143, 33-45.

Peng, Y., Li, H., Wu, P., Zhang, Y., Sun, X., & Wu, F. (2024). D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement. *arXiv preprint arXiv:2410.13842*

Perry, E.M. & Davenport, J.R. (2007). Spectral and spatial differences in response of vegetation indices to nitrogen treatments on apple. *Computers and Electronics in Agriculture*, 59(1-2), pp.56–65. doi:<https://doi.org/10.1016/j.compag.2007.05.002>.

Plantix (n.d.). Plantix | Best Agriculture App. [online] Plantix. Available at:
<https://plantix.net/en/>.

Pratap, V., Tjandra, A., Shi, B., Tomasello, P., Babu, A., Kundu, S., Elkahky, A., Ni, Z., Vyas, A., Fazel-Zarandi, M., Baevski, A., Adi, Y., Zhang, X., Hsu, W.-N., Conneau, A. and Auli, M. (2023). Scaling Speech Technology to 1,000+ Languages. *arXiv*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *Radford2019language*.

Rafel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. & Liu, P. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, (21), pp.1–67.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140), 1-67.

General Demographic Characteristics. (n.d.). Available at:

<https://www.knbs.or.ke/wp-content/uploads/2025/12/2019-KPHC-Atlas-General-Demographic-Characteristics.pdf>.

Rahim, A., Maqbool, A., & Rana, T. (2021). Monitoring social distancing under various low light conditions with deep learning and a single motionless time of flight camera. *PLOS ONE*. 16. e0247440. 10.1371/journal.pone.0247440.

Readthedocs.io. (2017). User Guide — Python Crop Simulation Environment 6.0 documentation. [online] Available at:

https://pcse.readthedocs.io/en/stable/user_guide.html#running-a-simulation-with-pcse-lintul3

Readthedocs.io. (2018). Code documentation — Python Crop Simulation Environment 6.0 documentation. [online] Available at: <https://pcse.readthedocs.io/en/stable/code.html>.

Readthedocs.io. (2022). pcse.input.yaml_crodataprovider — Python Crop Simulation Environment 6.0 documentation. [online] Available at:

https://pcse.readthedocs.io/en/stable/_modules/pcse/input/yaml_crodataprovider.html.

Readthedocs.io. (2024). pcse.input.sitedataproviders — Python Crop Simulation Environment 6.0 documentation. [online] Available at:

https://pcse.readthedocs.io/en/6.0.8/_modules/pcse/input/sitedataproviders.html.

Reeves, N. P., Sal, Y., Rosas, Celi, V. G., Lutomia, A. N., Medendorp, J. W., Bello-Bravo, J. & Pittendrigh, B. (2024) Agricultural education in Africa using YouTube multilingual animations: A retrospective feasibility study assessing costs to reach language-diverse populations. doi: 10.1371/journal.pone.0302136.

Saad, E., Elekyaby, M. S., Ali, E. O., & Hassan, S. F. A. E. (2020). Double diamond strategy saves time of the design process. *International Design Journal*, 10(3), 211-222.

Saha, H.N., Roy, R., Chakraborty, M. & Sarkar, C. (2021). Development of IoT-Based Smart Security and Monitoring Devices for Agriculture. In *Agricultural Informatics: Automation Using the IoT and Machine Learning*; John Wiley & Sons: Hoboken, NJ, USA,; pp. 147–169.

Saxton, K.E. & Rawls, W.J. (2006). Soil water characteristic estimates by texture and organic matter. *Soil Science Society of America Journal*, 70, 1569–1578

Schuermans, J., & Frasincar, F. (2019). Intent classification for dialogue utterances. *IEEE Intelligent Systems*, 35(1), 82-88.

Sim, W. S., Lim, K. Y., & Lim, T. M. (2025). Comparison of Machine Learning and Deep Learning Models for Intent Classification and Context Understanding. In *2025 IEEE*

International Conference on Computation, Big-Data and Engineering (ICCBDE) (pp. 335-340). IEEE.

Singh, A. (1994). CROP GROWTH SIMULATION MODELS. Water Technology Centre, I.A.R.I.

Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S. and Batra, N. (2019). PlantDoc: A Dataset for Visual Plant Disease Detection. arXiv (Cornell University). doi:<https://doi.org/10.48550/arxiv.1911.10317>.

Soegaard, M. (2025). System Usability Scale for Data-Driven UX. IxDF - Interaction Design Foundation. <https://www.interaction-design.org/literature/article/system-usability-scale>

Stackhouse, P. (2024). NASA POWER | Docs | Data Services | API Overview - NASA POWER | Docs. [online] Nasa.gov. Available at: <https://power.larc.nasa.gov/docs/services/api/>.

Statista. (2023). Kenya: food import dependency and independency 2023| Statista. [online] Available at: <https://www.statista.com/statistics/1154175/import-dependency-and-self-sufficiency-on-food-imports-in-kenya/?latest>.

Stroppiana, D., Boschetti, M., Brivio, P.A. & Bocchi, S. (2009). Plant nitrogen concentration in paddy rice from field canopy hyperspectral radiometry. Field Crops Research, 111(1-2), pp.119–129. doi:<https://doi.org/10.1016/j.fcr.2008.11.004>.

Sultana, F., Sufian, A., Dutta, P. (2020). A Review of Object Detection Models Based on Convolutional Neural Network. In: Mandal, J., Banerjee, S. (eds) Intelligent Computing: Image Processing Based Applications. Advances in Intelligent Systems and Computing, vol 1157. Springer, Singapore. https://doi.org/10.1007/978-981-15-4288-6_1

Trymata (2023). What is User Feedback? Definition, Types, Examples and Best Practices - Trymata. [online] Trymata. Available at: <https://trymata.com/blog/what-is-user-feedback/>.

UCI Machine Learning Repository. (2023). ucimlrepo package [Source Code]. UCI Machine Learning Repo. Available at: <https://github.com/uci-ml-repo/ucimlrepo>

Vaswani, A. (2017) Attention is all you need. Advances in Neural Information Processing Systems.

Verkaart, S., Mausach, K., & Harris, D. (2018). Who are those people we call farmers? Rural Kenyan aspirations and realities. Development in Practice, 28(4), 468–479. <https://doi.org/10.1080/09614524.2018.1446909>.

VOSK Offline Speech Recognition API. (n.d.). VOSK Offline Speech Recognition API. [online] Available at: <https://alphacephei.com/vosk/>.

W3C (2024). Web content accessibility guidelines (WCAG) 2.2. [online] www.w3.org. Available at: <https://www.w3.org/TR/WCAG22/>.

Wageningen University & Research. (2025). WOFOST - WOId FOod STudies. [online] Available at: <https://www.wur.nl/en/research/products-services/wofost-world-food-studies>.

Wallace, S., Bylinskii, Z., Dobres, J., Kerr, B., Berlow, S., Treitman, R., Kumawat, N., Arpin, K., Miller, D. B., Huang, J., & Sawyer, B. D. (2022). Towards Individuated Reading Experiences: Different Fonts Increase Reading Speed for Different Individuals. *ACM Transactions on Computer-Human Interaction*, 29(4), Article 38. <https://doi.org/10.1145/3502222>.

Wang, S., Xu, D., Liang, H., Bai, Y., Li, X., Zhou, J., Su, C., & Wei, W. (2025). Advances in Deep Learning Applications for Plant Disease and Pest Detection: A Review. *Remote Sensing*, 17(4), 698. <https://doi.org/10.3390/rs17040698>.

Weizenbaum, J. (1966). ELIZA - a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, [online] 9(1), pp.36–45. doi:<https://doi.org/10.1145/365153.365168>.

Wekesa, M. (2024). State pushes for fiber internet access to 8.5 million businesses. [online] Government Advertising Agency. Available at: <https://www.mygov.go.ke/state-pushes-fiber-internet-access-85-million-businesses>.

Wu, X., Zhan, C., Lai, Y.-K., Cheng, M.-M. and Yang, J. (2019). IP102: A Large-Scale Benchmark Dataset for Insect Pest Recognition. [online] IEEE Xplore. doi:<https://doi.org/10.1109/CVPR.2019.00899>.

Xarvio.com. (2025). FIELD MANAGER. [online] Available at: <https://www.xarvio.com/us/en/products/field-manager.html>.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., & Qiu, Z. (2025). Qwen3 technical report. arXiv preprint arXiv:2505.09388.

Yigit, G. & Bayraktar, R. (2025). Chatbot development strategies: a review of current studies and applications. *Knowl Inf Syst* 67, 7319–7354 . <https://doi.org/10.1007/s10115-025-02462-x>.

Zeghidour, N., Kharitonov, E., Orsini, M., Volhejn, V., de Marmiesse, G., Grave, E., Perez, P., Mazaré, L. and Défossez, A. (2026). kyutai/tts-1.6b-en_fr.

Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., Xie, C., & Zhou, Z. (2025). Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. arXiv preprint arXiv:2508.06471.

Zhang, J., Pu, R., Yuan, L., Wang, J., Huang, W. & Yang, G. (2014). Monitoring Powdery Mildew of Winter Wheat by Using Moderate Resolution Multi-Temporal Satellite Imagery. PLoS ONE, 9(4), p.e93107. doi:<https://doi.org/10.1371/journal.pone.0093107>.

Zhang, Y., Li, C., Zhou, X. & Moore, B. (2002). A simulation model linking crop growth and soil biogeochemistry for sustainable agriculture. Ecological Modelling, 151(1), pp.75–108. doi:[https://doi.org/10.1016/s0304-3800\(01\)00527-0](https://doi.org/10.1016/s0304-3800(01)00527-0).

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs Beat YOLOs on Real-time Object Detection. Proceedings (IEEE Computer Society Conference on Computer Vision and Pattern Recognition. Online), 16965–16974. <https://doi.org/10.1109/CVPR52733.2024.01605>

Appendices

Appendix A

Listing A.1

```
#Function for obtaining WOFOST soil data
def fetch_soil_parameters(longitude, latitude, filename, soil_name):
    properties = ["1500", "933", "bdod", "sand", "clay", "silt", "ocd"]
    prop_str "".join(properties)
    url =
    "https://rest.isric.org/soilgrids/v2.8/properties/query?lan={Longitude}&lat=
    {latitude}Sproperties={prop_str}&depth=8-Scn&value=sean"
    response = requests.get(url)

    if response.status_code==200:
        data response.json() parse JSON
    else:
        print(f"Error: {response.status_code}")
        data = None
        return

    soil_values = {}
    for layer in data["properties"]["layers"]:
        name = layer["name"]
        if name in properties:
            mean_val = layer["depths"][0]["values"]["mean"]
            soil_values[name] = mean_val

    #Handle missing values by setting defaults
    if not soil_values["bdod"]:
        soil_values["bdod"] = 1.5
    if not soil_values["sand"]:
        soil_values["sand"] = 25
    if not soil_values["silt"]:
        soil_values["silt"] = 25
    if not soil_values["clay"]:
        soil_values["clay"] = 25
    if not soil_values["wv0033"]:
        soil_values["wv0033"] = 0.30
    if not soil_values["wv1500"]:
        soil_values["wv1500"] = 0.15

    #Division by 100 and 1000 to get values in g/cm^3
    sm0 = (1-((soil_values["bded"]/100)/2.65))
    smfcf = soil_values["wv0033"] / 1000
    smw = soil_values["wv1500"]/1000
```

```

    #Use the most soil type in the soil
    if max(soil_values["sand"], soil_values["clay"], soil_values["silt"])
soil_values["sand"]:
        sope 9.8
    elif max(soil_values["sand"], soil_values["clay"], soil_values["silt"])
== soil_values["clay"]:
        sope 8.4
    else:
        sope 9.2

    craire 0.06
    k=25.586
    ksub = 1.47
    resel = 80
    spads = 0.10

```

Listing A.1: Python function for SoilGrids API data retrieval.

Listing A.2

```

# Gan, T. (2023). Jupyter Notebooks for the soilgrids Data Component,
HydroShare,
https://www.hydroshare.org/resource/7ad709d065274ad2a2a8fd08860513d5/
import matplotlib.pyplot as plt
from soilgrids import SoilGrids

def soilgrids_data(lat, lon):
    buffer = 0.005

    variables = {
        "bdod": "bdod_0-5cm_mean",
        "sand": "sand_0-5cm_mean",
        "clay": "clay_0-5cm_mean",
        "soc": "soc_0-5cm_mean",
        "cfvo": "cfvo_0-5cm_mean",
    }
    results = {}

    for variable in variables.keys():
        soil_grids = SoilGrids()
        data = soil_grids.get_coverage_data(
            service_id=variable,
            coverage_id=variables[variable],
            west=lon - buffer,
            south=lat - buffer,
            east=lon + buffer,
            north=lat + buffer,

```

```

        width=int(((lon+buffer) - (lon-buffer)) / 0.002245),
        height=int(((lat+buffer) - (lat-buffer)) / 0.002245),
        crs="urn:ogc:def:crs:EPSG::4326",
        local_file=False,
        output="test.tif"
    )
    # Some of the results tended to give negative values which are unusable
    # Default values i.e values obtained from Butere subcounty which was the
    first subcounty were used
    result = float(data.mean().values)
    if result < 0:
        if variable == "bdod":
            result = 120.75
        elif variable == "sand":
            result = 287.8125
        elif variable == "clay":
            result = 481.0
        elif variable == "soc":
            result = 422.25
        if variable == "cfvo":
            result = 114.4375

    results[variable] = result

return results

```

Listing A.2: Python function for SoilGrids Python Package data retrieval

Listing A.3

```

def soilgrids_site(lat, lon):
    buffer = 0.005

    variables = {
        "sand": "sand_0-5cm_mean",
        "clay": "clay_0-5cm_mean",
        "soc": "soc_0-5cm_mean",
    }
    results = {}

    for variable in variables.keys():
        soil_grids = SoilGrids()
        data = soil_grids.get_coverage_data(
            service_id=variable,
            coverage_id=variables[variable],
            west=lon - buffer,
            south=lat - buffer,
            east=lon + buffer,

```

```

        north=lat + buffer,
        width=int(((lon+buffer) - (lon-buffer)) / 0.002245),
        height=int(((lat+buffer) - (lat-buffer)) / 0.002245),
        crs="urn:ogc:def:crs:EPSG::4326",
        local_file=False,
        output="test.tif"
    )
    # Some of the results tended to give negative values which are unusable
    # Default values i.e values obtained from Butere subcounty which was the
    first subcounty were used
    result = float(data.mean().values)
    if result < 0:
        if variable == "sand":
            result = 287.8125
        elif variable == "clay":
            result = 481.0
        elif variable == "soc":
            result = 422.25

    results[variable] = result

    return results

```

Listing A.3: Function for obtaining site data for the Lintul model

Listing A.4

```

crop_data = YAMLCropDataProvider(Wofost71_PP)

def test_wofost(crop, variety, location, soil_file, latitude, longitude):
    # Crop data
    crop_data.set_active_crop(crop, variety)

    # Agromanagement data
    agromanagement_file = define_agromanagement(crop, variety, "2024-01-01",
"2024-12-31", f"/content/Agromanagement/{location}.agro")
    agromanagement = YAMLAGroManagementReader(agromanagement_file)

    # Soil data
    soil_data = CABOFileReader(soil_file)

    # Weather data
    weather_data = NASAPowerWeatherDataProvider(latitude, longitude)

    # dummy site data since data was not collected from an actual field
    sitedata = WOFOST72SiteDataProvider(WAV=100)

    # Set model params

```

```

    params = ParameterProvider(cropdata=crop_data, soildata=soil_data,
sitedata=sitedata)

    # run the model
    model = Wofost71_PP(params, weather_data, agromanagement)
    model.run_till_terminate()
    output = model.get_output()

    return output

```

Listing A.4: test_wofost function

Listing A.5

```

def test_lintul(crop_params, location, lat, lon, crop):
    crop = PCSEFileReader(fetch_crop_data("03-Dec-2006, 11:39:49",
"MIXED-DATA", crop_params, f"/content/lintul_crop/{location}.crop"))
    soil =
PCSEFileReader(f'/content/LINTUL_DATA/LINTUL_DATA/LINTUL_SITE/{location}_soi
l.site')
    site =
PCSEFileReader(f'/content/LINTUL_DATA/LINTUL_DATA/LINTUL_SOIL/{location}_soi
l.soil')

    parameterprovider = ParameterProvider(soildata=soil, cropdata=crop,
sitedata=site)

    weatherdataprovder = NASAPowerWeatherDataProvider(latitude=lat,
longitude=lon)

    agromanagement_file = define_agromanagement_lintul("maize", "maize",
"2024-01-01", "2024-10-20", "/content/agromanagement.amgt")
    agromanagement = YAMLAGroManagementReader(agromanagement_file)

    lintul3 = LINTUL3(parameterprovider, weatherdataprovder, agromanagement)
    lintul3.run_till_terminate()
    output = lintul3.get_output()

    return output

```

Listing A.5: test_lintul function

Listing A.6

```

def run_simulation():
    global intent
    global crop_sim_data

```



```

global pending_intent

planting_duration = {
    "barley": 6,
    "cassava": 13,
    "chickpea": 4,
    "cotton": 6,
    "cowpea": 4,
    "fababean": 5,
    "groundnut": 4,
    "maize": 6,
    "millet": 3,
    "mungbean": 3,
    "pigeonpea": 5,
    "potato": 4,
    "rapeseed": 3,
    "rice": 4,
    "sorghum": 7,
    "soybean": 4,
    "sugarbeet": 6,
    "sugarcane": 15,
    "sunflower": 4,
    "sweetpotato": 5,
    "tobacco": 4,
    "wheat": 5,
    "seed_onion": 4,
}

# Crop data
crop_data.set_active_crop(crop_sim_data['crop_name'],
crop_sim_data['crop_variety'])

# Agromanagement data
# Start date is assumed to be a year before the current day. End
date is calculated based on typical planting season length for the
particular crop
file_path = os.path.join(BASE_DIR, "data", "agromanagement.agro")
start_date = date.today() - relativedelta(years=1)
agromanagement_file =
define_agromanagement(crop_sim_data['crop_name'],
crop_sim_data['crop_variety'], start_date, start_date +
relativedelta(months=planting_duration[crop_sim_data['crop_name']]),
file_path)
agromanagement = YAMLAGroManagementReader(agromanagement_file)

# Soil data

```

```

    soil_file = os.path.join(BASE_DIR, "data", "soil",
f"{crop_sim_data['location']}.soil")
    soil_data = CABOFileReader(soil_file)

    # Weather data
    weather_data =
NASAPowerWeatherDataProvider(crop_sim_data['latitude'],
crop_sim_data['longitude'])

    sitedata = WOFOST72SiteDataProvider(WAV=100)

    params = ParameterProvider(cropdata=crop_data, soildata=soil_data,
sitedata=sitedata)

    # run the model
    model = Wofost71_PP(params, weather_data, agromanagement)
    model.run_till_terminate()

    summary = model.get_summary_output()[0]

    harvest_date = summary['DOM']
    yeild = summary['TWSO']

    output = model.get_output()
    total_transpiration = sum(day['TRA'] for day in output if day['TRA']
is not None)
    total_evaporation = sum(day['EVS'] for day in output if day['EVS']
is not None)

    total_water_use = total_transpiration + total_evaporation * 100000

    intent = None
    pending_intent = None
    del crop_sim_data['crop_name']
    del crop_sim_data['crop_variety']

    return f"Your expected harvest date is {harvest_date}. With optimal
conditions, you can expect a yeild of {yeild} per hectare. The total
amount of water you can expect to use is {total_water_use}"

```

Listing A.6: run_simulation function

Appendix B

Listing B.1

```
# Huggingface.co. (2017). RT-DETR. [online] Available at:
https://huggingface.co/docs/transformers/v4.42.0/model\_doc/rt\_detr.

# test Peking model
peking_model_results = []

for id in img_ids:
    # Select image
    imgIds = coco_gt.getImgIds(imgIds = [id])
    img =
    Image.open(requests.get(coco_gt.loadImgs(imgIds[np.random.randint(0, len(imgI
ds))])[0]['coco_url'], stream=True).raw).convert("RGB")

    processor =
    RTDetrImageProcessor.from_pretrained("PekingU/rt detr_r101vd_coco_o365")
    model =
    RTDetrForObjectDetection.from_pretrained("PekingU/rt detr_r101vd_coco_o365")

    input = processor(images=img, return_tensors="pt")
    outputs = model(**input)

    results = processor.post_process_object_detection(outputs,
target_sizes=torch.tensor([image.size[:-1]]), threshold=0.3)

    output = []

    for item in results:
        for s, l, b in zip(item["scores"], item["labels"], item["boxes"]):
            s, label = score.item(), l.item()
            box = xyxy_to_xywh([round(i, 2) for i in b.tolist()])

            if model.config.id2label[label] in name_to_id:
                peking_model_results.append({
                    "image_id": img['id'],
                    "category_id": name_to_id[model.config.id2label[label]],
                    "bbox": box,
                    "score": score
                })
```

Listing B.1: Peking model testing

Appendix C

Listing C.1

```
# Models that will be tested
models = {
    "SVM": SVC(kernel='linear'),
    "K-NN": KNeighborsClassifier(n_neighbors=5),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Multinomial NB": MultinomialNB(),
    "Random Forest": RandomForestClassifier(n_estimators=100)
}

param_grids = {
    "SVM": {
        'clf__C': [0.1, 1, 10],
        'clf__kernel': ['linear', 'rbf']
    },
    "K-NN": {
        'clf__n_neighbors': [3, 5, 11],
        'clf__weights': ['uniform', 'distance']
    },
    "Logistic Regression": {
        'clf__C': [0.1, 1, 10],
        'clf__penalty': ['l2']
    },
    "Multinomial NB": {
        'clf__alpha': [0.1, 0.5, 1.0]
    },
    "Random Forest": {
        'clf__n_estimators': [50, 100],
        'clf__max_depth': [None, 10, 20]
    }
}

# Train test split
X_train, X_test, y_train, y_test = train_test_split(imdb_df['review'],
imdb_df['sentiment'], test_size=0.2, random_state=42)

results = {}
best_models = {}

for name, model in models.items():
    # Construct the pipeline for each specific model with TFIDF
    vectorization
    pipeline = Pipeline([
```

```

        ('tfidf', TfidfVectorizer()),
        ('clf', model)
    ])

    grid_search = GridSearchCV(
        pipeline,
        param_grid=param_grids[name],
        cv=5,
        scoring='f1_weighted',
        n_jobs=-1 # Uses all CPU cores to speed up training
    )

    grid_search.fit(X_train, y_train)
    y_pred = grid_search.best_estimator_.predict(X_test)

    # Store the best estimator
    best_models[name] = grid_search.best_estimator_

    # Calculate each models accuracy, precision, recall and F1-score
    results[name] = {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred,
average='weighted'),
        "Recall": recall_score(y_test, y_pred, average='weighted'),
        "F1-Score": f1_score(y_test, y_pred, average='weighted')
    }

```

Listing C.1: Intent handling models testing

Listing C.2

```

import random

def augment_crop_data(raw_data, multiplier=2):
    crops = ["maize", "onion", "barley", "rice", "cassava", "wheat",
"soybean", "potato", "tomato"]
    synonyms = {
        "problem": ["issue", "concern", "trouble", "threat"],
        "normal": ["standard", "expected", "typical", "okay"],
        "worried": ["concerned", "anxious", "stressed"],
        "simulate": ["model", "predict", "forecast", "show me a
simulation of"],
    }

    augmented_dataset = list(raw_data) # Start with original data

    for _ in range(multiplier):

```

```

    for entry in raw_data:
        new_text = entry["text"]

        # Randomly swap crop names
        current_crops = [c for c in crops if c in new_text.lower()]
        if current_crops:
            for c in current_crops:
                new_text = new_text.replace(c, random.choice(crops))

        # Randomly replace keywords with synonyms
        for word, replacements in synonyms.items():
            if word in new_text.lower():
                new_text = new_text.replace(word,
random.choice(replacements))

        # Add small variations to "worry" phrases
        if entry["intent"] == "crop_growth_analysis" and "wrong" in
new_text:
            prefix = random.choice(["I think", "It looks like",
            "Actually,"])
            new_text = f"{prefix} {new_text}"

        augmented_dataset.append({"intent": entry["intent"], "text":
new_text})

    return augmented_dataset

# Storing the result in the variable 'dataset'
dataset = augment_crop_data(raw_data, multiplier=3)

```

Listing C.2: Intent handling data augmentation

Appendix D

Listing D.1

```

import torch

all_predictions = {
    'qwen_bertscore_results_precision': 0,
    'qwen_bertscore_results_recall': 0,
    'qwen_bertscore_results_f1': 0,
    'wer_qwen': 0,
    'toxicity': []
}

```

```

for item in df:
    prompt = item['prompt']
    # Huggingface.co. (2026). Qwen/Qwen2.5-1.5B-Instruct · Hugging Face.
    [online] Available at:
    https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct?library=transformers.
    message = [
        {"role": "system", "content": "You are Qwen, created by Alibaba Cloud.
    You are a helpful assistant."},
        {"role": "user", "content": prompt}
    ]

    text_input = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )
    inputs = tokenizer([text_input], return_tensors="pt").to(model.device)

    ids = model.generate(
        **inputs,
        max_new_tokens=512
    )

    for input_ids, output_ids in zip(inputs.input_ids, generated_ids):
        ids.append(output_ids[len(input_ids):])

    response = tokenizer.batch_decode(ids, skip_special_tokens=True)[0]

    reference = item['response_a'] if item['winner'] == 'winner_model_a' else
    item['response_b']

    # Calculate BERTScore
    qwen_bertscore_results = bertscore.compute(predictions=[response],
    references=[reference], lang="en")
    all_predictions['qwen_bertscore_results_precision'] +=
    qwen_bertscore_results['precision']
    all_predictions['qwen_bertscore_results_recall'] +=
    qwen_bertscore_results['recall']
    all_predictions['qwen_bertscore_results_f1'] +=
    qwen_bertscore_results['f1']

    # Calculate WER
    all_predictions['wer_qwen'] += wer_metric.compute(predictions=[response],
    references=[reference])
    all_predictions['toxicity'].append(response)

# Calculate toxicity

```

```

toxicity_qwen_results = toxicity_metric.compute(predictions=[response])

qwen_bertscore_results = all_predictions['qwen_bertscore_results_precision']
/ 44630
qwen_bertscore_results = all_predictions['qwen_bertscore_results_recall'] /
44630
qwen_bertscore_results = all_predictions['qwen_bertscore_results_f1'] /
44630
wer_qwen = all_predictions['wer_qwen'] / 44630

toxicity_qwen_results =
toxicity_metric.compute(predictions=all_predictions['toxicity'])

# Print results
print(f"BERTScore Precision: {qwen_bertscore_results:.4f}")
print(f"BERTScore Recall: {qwen_bertscore_results:.4f}")
print(f"BERTScore F1: {qwen_bertscore_results:.4f}")
print(f"WER: {wer_qwen:.4f}")

print(f"Qwen Model Toxicity Score: {toxicity_qwen_results['toxicity']}")

```

Listing D.1: Testing for the Qwen generation model

Appendix E

Listing E.1

```

# Huggingface.co. (2026). AventIQ-AI/whisper-audio-to-text · Hugging Face.
[online] Available at:
https://huggingface.co/AventIQ-AI/whisper-audio-to-text.

def load_audio(file_path, target_sampling_rate=16000):
    # Load audio file
    waveform, sample_rate = torchaudio.load(file_path)

    # Convert to mono if stereo
    if waveform.shape[0] > 1:
        waveform = waveform.mean(dim=0, keepdim=True)

    # Resample if needed
    if sample_rate != target_sampling_rate:
        waveform = torchaudio.transforms.Resample(orig_freq=sample_rate,
new_freq=target_sampling_rate)(waveform)

    return waveform.squeeze(0).numpy()
avent_wer = 0
avent_cer = 0

```



```

aevent_toxicity = []

for i, item in ds['train']: # enumerate(ds['train'].select(range(1)))
    # Get audio data directly from the dataset entry
    audio_data = item['audio']
    original_audio_array = audio_data['array']
    original_sampling_rate = audio_data['sampling_rate']

    target_sampling_rate = 16000

    # Convert original_audio_array (numpy) to a torch tensor for torchaudio
    operations
    # Add a channel dimension (batch_size, channels, samples) and convert to
    float
    waveform = torch.from_numpy(original_audio_array).unsqueeze(0).float()

    # Resample if needed
    if original_sampling_rate != target_sampling_rate:
        resampler =
torchaudio.transforms.Resample(orig_freq=original_sampling_rate,
new_freq=target_sampling_rate)
        waveform = resampler(waveform)

    # Convert back to numpy and remove the channel dimension
    audio_array_processed = waveform.squeeze(0).numpy()

    input_features = processor(audio_array_processed,
sampling_rate=target_sampling_rate, return_tensors="pt").input_features
    input_features = input_features.to(device)

    forced_decoder_ids = processor.get_decoder_prompt_ids(language="en",
task="transcribe")

    with torch.no_grad():
        predicted_ids = model.generate(input_features,
forced_decoder_ids=forced_decoder_ids)

    # Decode output
    transcription = processor.batch_decode(predicted_ids,
skip_special_tokens=True)[0]
    reference = item['text']

    aevent_wer += wer_metric.compute(predictions=[transcription],
references=[reference])
    aevent_cer += cer_metric.compute(predictions=[transcription],
references=[reference])
    aevent_toxicity.append(transcription)

```

```

avent_wer = avent_wer / 1000
avent_cer = avent_cer / 1000
avent_toxicity =
toxicity_metric.compute(predictions=avent_toxicity)['toxicity'][0]

print(f"WER Score: {avent_wer:.4f}")
print(f"CER Score: {avent_cer:.4f}")
print(f"Toxicity Score: {avent_toxicity:.4f}")

```

Listing E.1: An example of Speech recognition models testing

Appendix F

Listing F.1

```

try:
    base64_string = tts_client.predict(
        text=reply,
        voice="af_heart",
        api_name="/generate_speech_as_bytes"
    )
    audio_data = base64.b64decode(base64_string)

    with open(generated_audio_path, "wb") as f:
        f.write(audio_data)
        print(generated_audio_path)

except Exception as e:
    print(e)

```

Listing F.1: Kokoro model access code

Appendix G

Listing G.1

```

@app.post("/signup")
def signup(data: UserSignup):
    global crop_sim_data

    try:
        # Check if username already exists
        existing = users_collection.find_one({"username":
data.username})
        if existing:

```

```

        raise HTTPException(400, "Username already exists")

    user_doc = {
        "username": data.username,
        "password_hash": hash_password(data.password),
        "input_type": data.input_type,
        "subcounty": data.subcounty
    }

    users_collection.insert_one(user_doc)

    crop_sim_data['location'] = data.subcounty
    crop_sim_data['latitude'] =
subcounty_lats[subcounties.index(crop_sim_data['location'])]
    crop_sim_data['longitude'] =
subcounty_lats[subcounties.index(crop_sim_data['location'])]

    # Create authentication token to save login state
    token = jwt.encode(
        {
            "sub": data.username,
            "exp": datetime.utcnow() + timedelta(days=7)
        },
        SECRET_KEY,
        algorithm=ALGORITHM
    )

    return {"access_token": token, "status": "success", "message":
    "User registered successfully"}
    except Exception as e:
        raise HTTPException(500, f"Signup failed: {str(e)}")

```

Listing G.1: Signup function

Listing G.2

```

@app.post("/login")
def login(data: UserLogin):
    global crop_sim_data

    try:
        user = users_collection.find_one({"username": data.username})

        if not user or not verify_password(data.password,
user["password_hash"]): # Check that the correct password has been given
            raise HTTPException(401, "Invalid credentials")

```

```

        if user['subcounty'] not in subcounties:
            raise HTTPException(status_code=400, detail="Invalid stored
subcounty")

        crop_sim_data['location'] = user['subcounty']
        crop_sim_data['latitude'] =
subcounty_lats[subcounties.index(crop_sim_data['location'])]
        crop_sim_data['longitude'] =
subcounty_lons[subcounties.index(crop_sim_data['location'])]

        # Create authentication token to save login state
        token = jwt.encode(
            {
                "sub": user['username'],
                "exp": datetime.utcnow() + timedelta(days=7)
            },
            SECRET_KEY,
            algorithm=ALGORITHM
        )

        return {"access_token": token, "input_type": user['input_type'],
"subcounty": user['subcounty']}
    except Exception as e:
        raise HTTPException(500, f"Login failed: {str(e)}")

```

Listing G.2: Login function

Listing G.3

```

@app.post("/message")
def handle_message(data: UserMessage):
    reply = handle_intent(data.message.lower())
    return {"reply": reply}

```

Listing G.3: handle_message function

Listing G.4

```

@app.post("/image")
def handle_image(imageFile: UploadFile = File(...),):
    global IMAGE
    global intent

    try:
        # Create uploads folder if it doesn't exist
        os.makedirs('uploads', exist_ok=True)

```

```

    IMAGE = os.path.join('uploads', imageFile.filename)

    # Save the file using shutil
    with open(IMAGE, "wb") as buffer:
        shutil.copyfileobj(imageFile.file, buffer)

    intent = 'crop_growth_analysis'
    reply = handle_intent('')

    return {"message": "Success", "reply": reply}

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

```

Listing G.4: handle_image function

Listing G.5

```

1. @app.post("/audio")
2. def handle_audio(audio_file: UploadFile = File(...)):
3.     if not audio_file.filename.endswith(('.3gp', '.wav', '.mp3')):
4.         raise HTTPException(status_code=400, detail="Unsupported
    audio format")
5.
6.     try:
7.         load_audio_models()
8.     except Exception as e:
9.         raise HTTPException(500, f"Error loading audio models:
    {str(e)}")
10.
11.     try:
12.         os.makedirs('uploads', exist_ok=True)
13.         temp_input_path = os.path.join('uploads',
    audio_file.filename)
14.         wav_path = os.path.join('uploads',
    f"conv_{int(time.time())}.wav")
15.
16.         with open(temp_input_path, "wb+") as file_object:
17.             shutil.copyfileobj(audio_file.file, file_object)
18.
19.         # Convert to WAV
20.         subprocess.run(
21.             [
22.                 "ffmpeg",
23.                 "-y",

```

```

24.         "-i", temp_input_path,
25.         "-ar", "16000",          # sample rate
26.         "-ac", "1",             # mono
27.         "-f", "wav",
28.         wav_path
29.     ],
30.     check=True
31. )
32.
33.     # Convert audio to text
34.     with wave.open(wav_path, "rb") as audio:
35.         rec = KaldiRecognizer(att_model, 16000)
36.         all_frames = audio.readframes(audio.getnframes())
37.         rec.AcceptWaveform(all_frames)
38.         result_text = json.loads(rec.FinalResult())["text"]
39.
40.         reply = handle_intent(result_text)
41.
42.         base64_string = tts_client.predict(
43.             text=reply,
44.             voice="af_heart",
45.             api_name="/generate_speech_as_bytes"
46.         )
47.
48.         return {"reply": reply, "audio": f"{base64_string}"}

```

Listing G.5: handle_audio function

Listing G.6

```

1. def handle_intent(text):
2.     global intent
3.     global crop_sim_data
4.     global pending_intent
5.
6.     if not intent:
7.         intent = intent_model.predict([text])[0]
8.
9.     if intent == "crop_simulation" or pending_intent == 'crop
simulation': # Pending intent will only be passed here on the
second call to crop simulation
10.         needed = get_simulation_data(text, crop_sim_data)
11.
12.         if len(needed) > 0:
13.             if not pending_intent:
14.                 reply = f"Please provide the following: {",
".join(needed)}. If you already provided a crop name, then we are

```

```

        sorry but it seems like we do not support that crop"
15.         pending_intent = "crop simulation"
16.         intent = None
17.     else:
18.         pending_intent = None
19.         intent = None
20.         reply = f"Please repeat that"
21.     else:
22.         reply = run_simulation()
23.
24.     return reply
25.
26.     elif intent == "crop_growth_analysis" or pending_intent ==
'crop_growth_analysis':
27.         if not IMAGE:
28.             if not pending_intent:
29.                 reply = "Please provide an image"
30.                 pending_intent = "crop_growth_analysis"
31.                 intent = None
32.             else:
33.                 pending_intent = None
34.                 intent = None
35.                 reply = f"Please repeat that"
36.         else:
37.             reply = analyze_image()
38.
39.         return reply
40.     else:
41.         intent = None
42.         reply = llm_client.chat.completions.create(
43.             model="jinaai/ReaderLM-v2:featherless-ai",
44.             messages=[
45.                 {
46.                     "role": "user",
47.                     "content": text
48.                 }
49.             ],
50.             ).choices[0].message.content
51.
52.     return reply

```

Listing G.6: handle_intent function

Listing G.7

```

def get_simulation_data(text, crop_sim_data):
    # Get crop

```

```

needed = []
possible_crops = list(crop_data.get_crops_varieties().keys())

crop_name = ""
crop_variety = ""

if 'crop_name' not in crop_sim_data.keys():
    for crop in possible_crops:
        if crop in text:
            crop_name = crop
            crop_variety =
list(crop_data.get_crops_varieties()[crop_name])[0]
else:
    crop_name = crop_sim_data['crop_name']
    crop_variety = crop_sim_data['crop_variety']

if crop_name == "":
    needed.append('Crop name')
else:
    crop_sim_data['crop_name'] = crop_name
    crop_sim_data['crop_variety'] = crop_variety

return needed

```

Listing G.7: get_simulation_data function

Listing G.8

```

def analyze_image():
    global intent
    global IMAGE

    intent = None
    issues = []

    diseases = client.image_classification(IMAGE,
model="linkanjarad/mobilenet_v2_1.0_224-plant-disease-identification")[0]
]['label']

    try:
        pests = pest_client.predict(
            img=handle_file(IMAGE),
            api_name="/predict_pest"
        )

        raw_list = ast.literal_eval(pests)
        pests = list(dict.fromkeys(raw_list))

```



```

        issues.extend(pests)
    except Exception as e:
        print(e)
        return "There was an error generating pest response. Please try
again later"

    if diseases.split(' ')[0].lower() != 'healthy':
        issues.append(diseases)

    # 1. Define the starting phrase
    base_string = "The following issues were identified in your crop: "

    # 2. Create a list of formatted strings for each key-value pair
    issue_descriptions = [f"Issue: {issue}, Treatment:
{treatments[issue]}" for issue in issues]

    try:
        os.remove(IMAGE)
        IMAGE = None
    except Exception as e:
        print("Failed to delete image:", e)

    if len(issues) > 0:
        return base_string + ", ".join(issue_descriptions) + "."
    else:
        return "Your crops seem fine"

```

Listing G.8: analyze_image function

Appendix H

Listing H.1

```

1. # reusable background template for a "card"
2. <CardTemplate@BoxLayout>:
3.     is_login_mode: False
4.     toggle_mode: lambda *args: None
5.     orientation: 'vertical'
6.     size_hint: (None, None)
7.     size: (400, 500)
8.     pos_hint: {'center_x': 0.5, 'center_y': 0.5}
9.     padding: 20
10.     canvas.before:
11.         Color:
12.             rgba: (90/255), (150/255), (63/255), 1
13.         RoundedRectangle:

```

```
14.         size: self.size
15.         pos: self.pos
16.         radius: [20, 20, 20, 20]
17.
18.     # User name input
19.     Label:
20.         text: "Enter Username:"
21.         color: 0, 0, 0, 1
22.         size_hint_y: None
23.         height: 30
24.         text_size: self.size
25.         halign: 'left'
26.         valign: 'middle'
27.         padding_x: 35
28.
29.     FocusTextInput:
30.         id: username_input
31.         multiline: False
32.         focus: True if
33.             self.collide_point(*self.parent.to_window(*self.pos)) else False
34.         size_hint_y: None
35.         height: 30
36.         size_hint_x: 0.8
37.         pos_hint: {'center_x': 0.5}
38.
39.     # Spacer
40.     Widget:
41.         size_hint_y: 0.05
42.
43.     # Password input
44.     Label:
45.         text: "Enter Password:"
46.         color: 0, 0, 0, 1
47.         size_hint_y: None
48.         height: 30
49.         text_size: self.size
50.         halign: 'left'
51.         valign: 'middle'
52.         padding_x: 35
53.
54.     FocusTextInput:
55.         id: password_input
56.         multiline: False
57.         focus: True if
58.             self.collide_point(*self.parent.to_window(*self.pos)) else False
59.         size_hint_y: None
```

```
58.         height: 30
59.         size_hint_x: 0.8
60.         pos_hint: {'center_x': 0.5}
61.
62.     Widget:
63.         size_hint_y: 0.05
64.
65.     BoxLayout:
66.         id: input_type_container
67.         orientation: "vertical"
68.         size_hint_y: None
69.         height: 0 if root.is_login_mode else 70
70.         opacity: 0 if root.is_login_mode else 1
71.         disabled: root.is_login_mode
72.         spacing: 5
73.
74.         Label:
75.             text: "Select your preferred input type"
76.             color: 0, 0, 0, 1
77.             size_hint_y: None
78.             height: 30
79.             text_size: self.size
80.             halign: 'left'
81.             valign: 'middle'
82.             padding_x: 35
83.
84.         BoxLayout:
85.             id: input_type_row
86.             orientation: "horizontal"
87.             size_hint_y: None
88.             height: 30
89.             size_hint_x: 0.8
90.             pos_hint: {'center_x': 0.5}
91.
92.             Label:
93.                 text: "Audio"
94.                 color: 0,0,0,1
95.             CheckBox:
96.                 id: audio_check
97.                 group: "input"
98.             Label:
99.                 text: "Text"
100.                 color: 0,0,0,1
101.             CheckBox:
102.                 id: text_check
103.                 group: "input"
```

```

104.
105.     Widget:
106.         size_hint_y: 0.1
107.
108.     MDBoxLayout:
109.         id: dropdown_menu
110.         orientation: "vertical"
111.         size_hint_x: 0.8
112.         size_hint_y: None
113.         height: 70 if not root.is_login_mode else 0
114.         pos_hint: {'center_x': 0.5}
115.         opacity: 1 if not root.is_login_mode else 0
116.         disabled: root.is_login_mode
117.         on_touch_down:
118.             if self.collide_point(*args[1].pos):
119.                 root.parent.parent.open_subcounty_menu()
120.
121.         MDTextField:
122.             id: subcounty_field
123.             hint_text: "Select subcounty"
124.             mode: "rectangle"
125.             readonly: True
126.
127.     # Spacer (pushes the card down)
128.     Widget:
129.         size_hint_y: 0.05
130.
131.     Label:
132.         text: root.parent.parent.error_message
133.         color: 1, 0, 0, 1
134.         size_hint_y: None
135.         height: 30
136.         text_size: self.size
137.         halign: "center"
138.         valign: "middle"
139.
140.     # Send Button
141.     Button:
142.         id: send_button
143.         text: "SEND"
144.         size_hint: (0.5, None)
145.         height: 50
146.         pos_hint: {'center_x': 0.5}
147.         background_color: 0,0,0,0
148.         background_normal: ''

```

```

149.         on_press:
150.             root.parent.parent.process_signup() if not
               root.is_login_mode \
151.             else root.parent.parent.process_login()
152.         canvas.before:
153.             Color:
154.                 rgba: (179/255, 156/255, 77/255, 1)
               RoundedRectangle:
                   size: self.size
                   pos: self.pos
                   radius: [15]

        # Spacer
        Widget:
            size_hint_y: 0.1

        Button:
            id: toggle_mode_button
            background_color: 0,0,0,0
            color: 0, 0, 1, 1 # Blue link color
            text:
                "Already have an account? Login here" if not
root.is_login_mode \
                else "Don't have an account? Signup here"
            on_release:
                root.toggle_mode()

```

Listing H.1: CardTemplate widget

Listing H.2

```

if audio_response.status_code == 200:
    response_path = self._build_response_audio_path()
    with open(response_path, "wb") as f:
        f.write(audio_response.content)
    self.response_audio_path = response_path
    self.ids.play_response.disabled = False
    self.ids.play_response.text = "Play response"

```

Listing H.2: Code for enabling play response button when audio is generated

Listing H.3

The source code can be found this github link:

<https://github.com/Hannah-Muthoni-Hailu/FarmersBot-AI-Assistant>